CSA1412-Compiler Design Lab Experiments

Name: Arava Raja **Reg. No.:192365033**

1. The lexical analyzer should ignore redundant spaces, tabs, and new lines. It should also ignore comments. Although the syntax specification states that identifiers can be arbitrarily long, you may restrict the length to some reasonable value. Develop a lexical Analyzer to identify identifiers, constants, and operators using the C program.

Aim:

To develop a lexical analyzer in C that identifies identifiers, constants, and operators from a given source program while ignoring redundant spaces, tabs, new lines, and comments.

Short Procedure:

1. Write the C program for the lexical analyzer.
2. Read the input source program character by character.
3. Ignore spaces, tabs, new lines, and comments.
4. Check each token and classify it as an identifier, constant, or operator.
5. Display the identified tokens on the screen.
6. Compile and execute the program.
7. Verify the output using a sample input program.

Code:

```
#include <stdio.h>
#include <ctype.h>
#include <string.h>
int isOperator(char ch)
{
   char operators[] = "+-*/%=<>!&|^";
   for (int i = 0; operators[i] != '\0'; i++)
   {
      if (ch == operators[i])
         return 1;
```

```
    }
    return 0;
}
int main()
{
    FILE *fp;
    char ch, buffer[100];
    int i;
    fp = fopen("input.c", "r");

    if (fp == NULL)
    {
        printf("Cannot open file.\n");
        return 1;
    }

    while ((ch = fgetc(fp)) != EOF)
    {
        // Ignore spaces, tabs and newlines
        if (isspace(ch))
            continue;

        // Ignore comments
        if (ch == '/')
        {
            char next = fgetc(fp);

            if (next == '/')
            {
                while ((ch = fgetc(fp)) != '\n' && ch != EOF);
                continue;
            }
            else if (next == '*')
            {
                char prev = 0;
                while ((ch = fgetc(fp)) != EOF)
                {
                    if (prev == '*' && ch == '/')
                        break;
                    prev = ch;
                }
                continue;
            }
            else
```

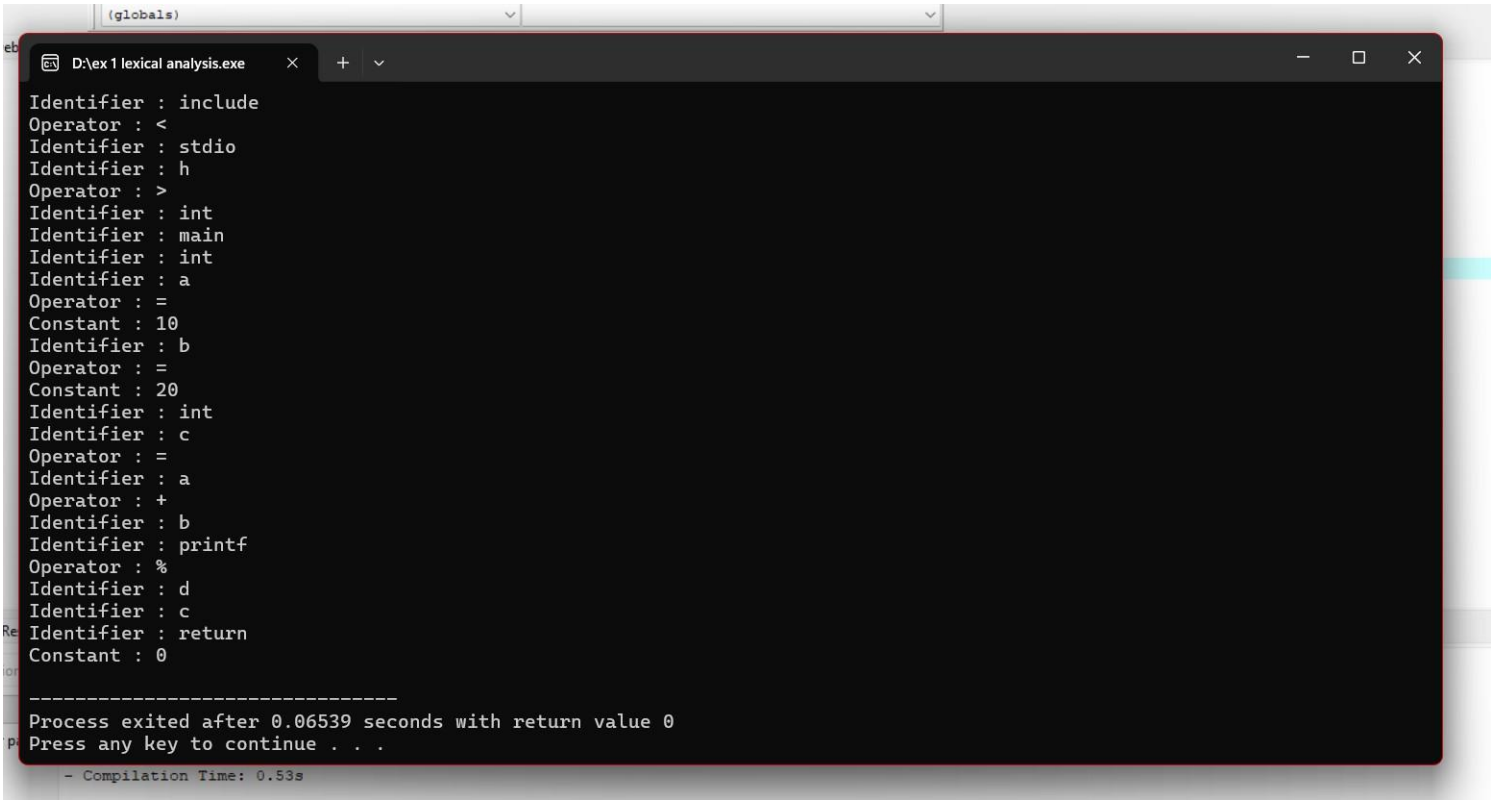
```
    {
        ungetc(next, fp);
        printf("Operator : %c\n", ch);
        continue;
    }
}

// Identifier
if (isalpha(ch) || ch == '_')
{
    i = 0;
    buffer[i++] = ch;

    while ((ch = fgetc(fp)) != EOF &&
           (isalnum(ch) || ch == '_'))
    {
        buffer[i++] = ch;
    }

    buffer[i] = '\0';
    printf("Identifier : %s\n", buffer);

    if (ch != EOF)
        ungetc(ch, fp);
}

// Constant
else if (isdigit(ch))
{
    i = 0;
    buffer[i++] = ch;

    while ((ch = fgetc(fp)) != EOF &&
           isdigit(ch))
    {
        buffer[i++] = ch;
    }

    buffer[i] = '\0';
    printf("Constant : %s\n", buffer);

    if (ch != EOF)
        ungetc(ch, fp);
}
```

```
        // Operator
        else if (isOperator(ch))
        {
            printf("Operator : %c\n", ch);
        }
    }

    fclose(fp);
    return 0;
}
```

Sample Input:

```
#include<stdio.h>
int main()
{
    int a = 10, b = 20;
    // Addition
    int c = a + b;
    /* Display result */
    printf("%d", c);
    return 0;
}
```

Output:



2. Extend the lexical Analyzer to Check comments, defined as follows in C:
a) A comment begins with // and includes all characters until the end of that line.
b) A comment begins with /* and includes all characters through the next occurrence of the character sequence */Develop a lexical Analyzer to identify whether a given line is a comment or not.

Aim:

To develop a lexical analyzer using Flex to identify whether a given input is a valid C comment. The analyzer should recognize both single-line (//) and multi-line (/* ... */) comments.

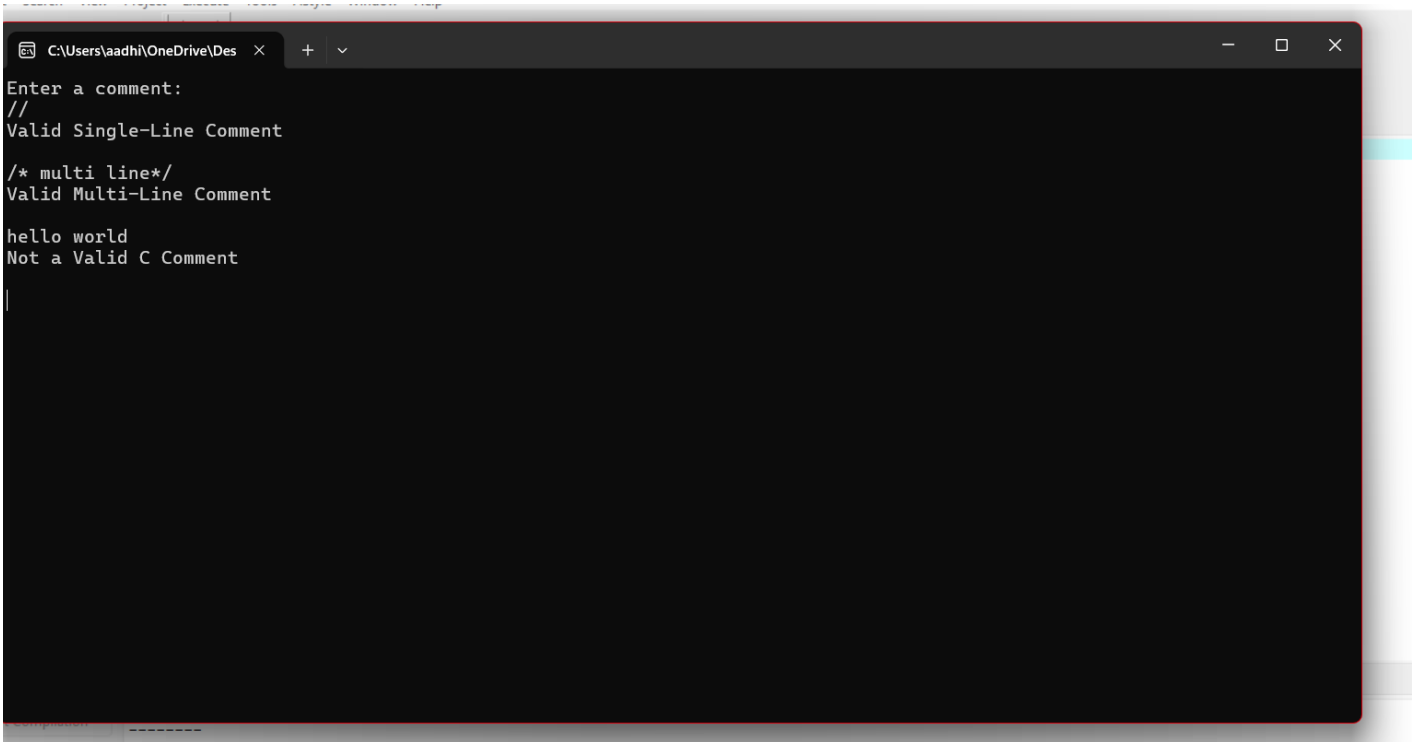
Short Procedure:

1. Write the Flex program (comment.l) with rules to detect // and /* ... */ comments.
2. Save the program and generate the lexical analyzer using the Flex command.
3. Compile the generated C file using GCC.
4. Execute the program and enter the input.
5. The lexical analyzer checks whether the input is a valid C comment.
6. Display "Valid Comment" if the input is a comment; otherwise display "Not a Comment".

Code:

```
%{
#include <stdio.h>
%}

%%

"//".*                    { printf("Single Line Comment\n"); }

"/*"([^*]|\*+[^*/])*\*+"/"  { printf("Multi Line Comment\n"); }

(.|\n)                    { printf("Not a Comment\n"); }

%%

int yywrap()
{
   return 1;
}

int main()
{
     printf("Enter input:\n");
     yylex();
     return 0;
  }
```

Output:



3. Design a lexical Analyzer for a given language should ignore the redundant spaces, tabs, and new lines and ignore comments**.**

Aim:

To design and implement a lexical analyzer using Flex that ignores redundant spaces, tabs, new lines, and comments in a given source program while processing the remaining valid tokens.

Short Procedure:

1. Write a Flex program (lexer.l) to define rules for ignoring spaces, tabs, new lines, and C comments (// and /* ... */).
2. Save the program and generate the lexical analyzer using the Flex command.
3. Compile the generated C source file using GCC.
4. Execute the program and provide the source code as input.
5. The lexical analyzer skips all unnecessary white spaces and comments.
6. The remaining valid tokens are processed and displayed as output.

Code:

%{

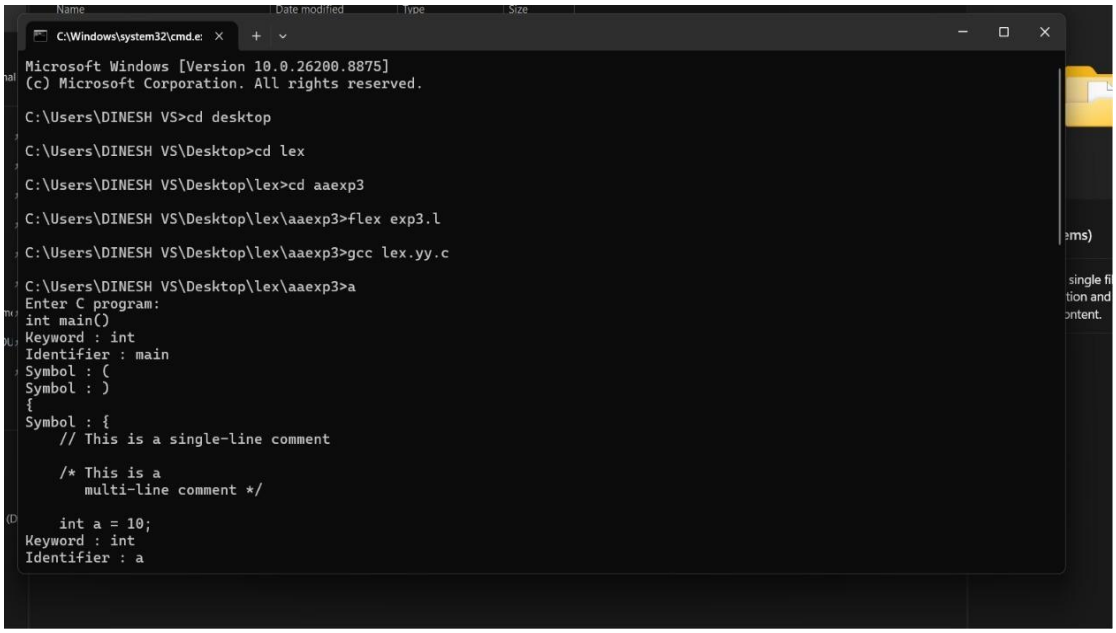
```
#include<stdio.h>
%}

/* Definitions */
DIGIT      [0-9]
LETTER     [a-zA-Z]
ID         {LETTER}({LETTER}|{DIGIT})*

%%

[ \t\n]+                    ;   /* Ignore spaces, tabs and new lines */

"//".*                      ;   /* Ignore single-line comments */

"/*"([^*]|\*+[^*/])*"*/"          ;   /* Ignore multi-line comments */

"int"                       { printf("Keyword : %s\n", yytext); }

"float"                     { printf("Keyword : %s\n", yytext); }

"char"                      { printf("Keyword : %s\n", yytext); }

"if"                        { printf("Keyword : %s\n", yytext); }

"else"                      { printf("Keyword : %s\n", yytext); }

{ID}                        { printf("Identifier : %s\n", yytext); }

{DIGIT}+                    { printf("Number : %s\n", yytext); }

"+"|"-"|"*"|"/"|"="         { printf("Operator : %s\n", yytext); }

";"|","|"("|")"|"{"|"}"     { printf("Symbol : %s\n", yytext); }

.                           { printf("Unknown : %s\n", yytext); }

%%

int yywrap()
{
   return 1;
}

int main()
```

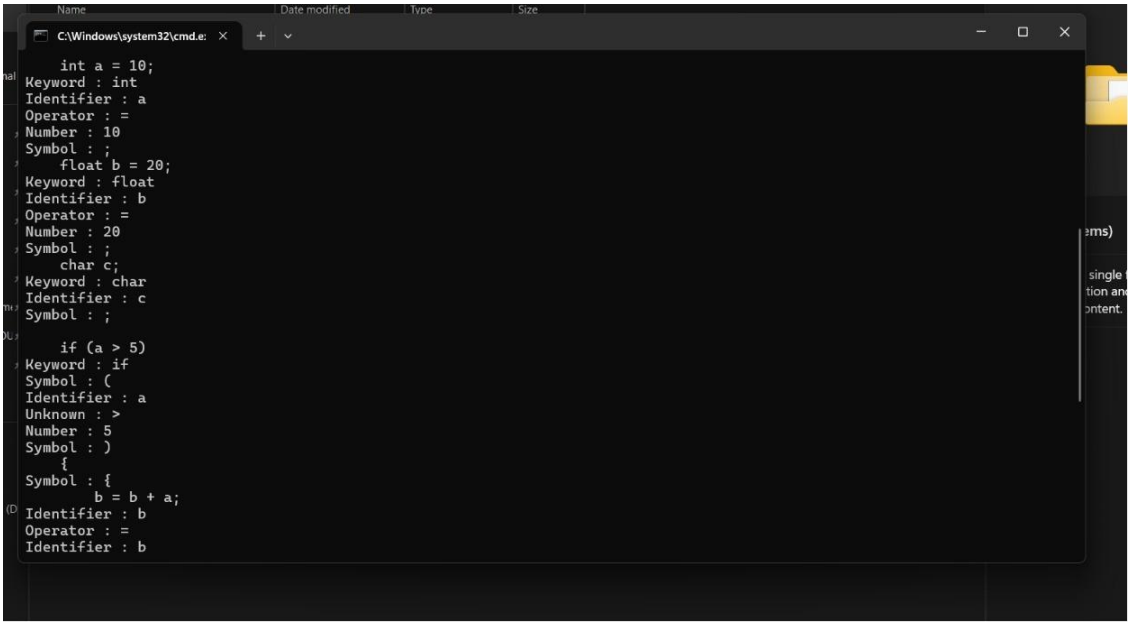
```
{
    printf("Enter C program:\n");
    yylex();
    return 0;
}
```

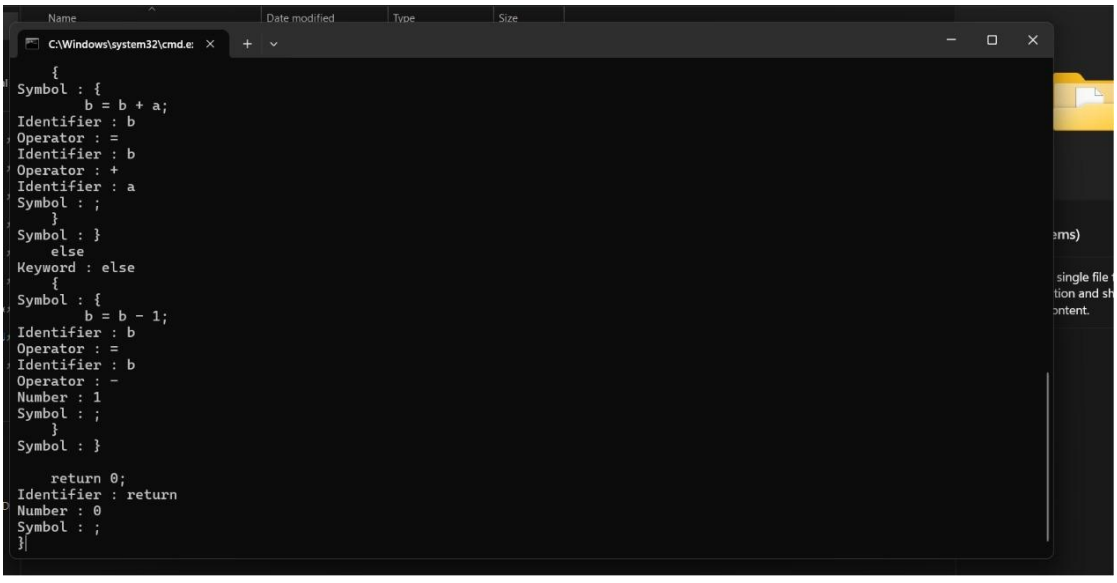
Sample Input:

```
int main()
{
    // This is a single-line comment

    /* This is a
       multi-line comment */

    int a = 10;
    float b = 20;
    char c;

    if (a > 5)
    {
        b = b + a;
    }
    else
    {
        b = b - 1;
    }

    return 0;
}
```

Output:


C:\Windows\system32\cmd.e:
Microsoft Windows [Version 10.0.26200.8875]
(c) Microsoft Corporation. All rights reserved.
C:\Users\DINESH VS>cd desktop
C:\Users\DINESH VS\Desktop>cd lex
C:\Users\DINESH VS\Desktop\lex>cd aaexp3
C:\Users\DINESH VS\Desktop\lex\aaexp3>flex exp3.l
C:\Users\DINESH VS\Desktop\lex\aaexp3>gcc lex.yy.c
C:\Users\DINESH VS\Desktop\lex\aaexp3>a
Enter C program:
int main()
Keyword : int
Identifier : main
Symbol : (
Symbol :)
{
Symbol : {
// This is a single-line comment
/* This is a
multi-line comment */
int a = 10;
Keyword : int
Identifier : a


C:\Windows\system32\cmd.e:
int a = 10;
Keyword : int
Identifier : a
Operator : =
Number : 10
Symbol : ;
float b = 20;
Keyword : float
Identifier : b
Operator : =
Number : 20
Symbol : ;
char c;
Keyword : char
Identifier : c
Symbol : ;
if (a > 5)
Keyword : if
Symbol : (
Identifier : a
Unknown : >
Number : 5
Symbol :)
{
Symbol : {
b = b + a;
Identifier : b
Operator : =
Identifier : b


C:\Windows\system32\cmd.e:
{
Symbol : {
b = b + a;
Identifier : b
Operator : =
Identifier : b
Operator : +
Identifier : a
Symbol : ;
}
Symbol : }
else
Keyword : else
{
Symbol : {
b = b - 1;
Identifier : b
Operator : =
Identifier : b
Operator : -
Number : 1
Symbol : ;
}
Symbol : }
return 0;
Identifier : return
Number : 0
Symbol : ;
}

4. Design a lexical Analyzer to validate operators to recognize the operators +,-,*,/ using regular Arithmetic operators.

Aim:

To design and implement a lexical analyzer using **Flex** to recognize and validate arithmetic operators (+, -, *, /) in the given input.

Short Procedure:

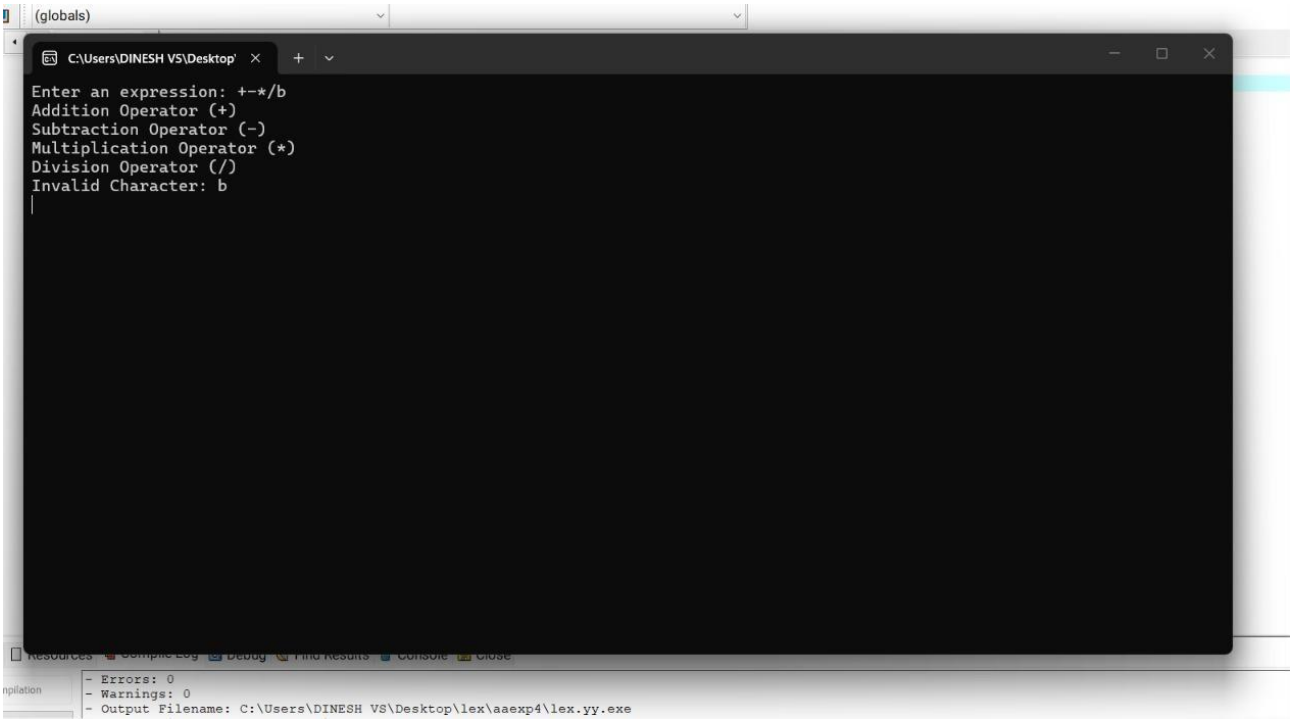
1. Write a Flex program (operator.l) with regular expressions to identify the arithmetic operators +, -, *, and /.
2. Save the program and generate the lexical analyzer using the **Flex** command.
3. Compile the generated C file using **GCC**.
4. Execute the program and provide the input.
5. The lexical analyzer checks whether the input contains valid arithmetic operators.
6. Display the recognized operator or indicate if the input is not a valid arithmetic operator.

Code:

```
%{
#include <stdio.h>
%}

%%

"+"     { printf("Addition Operator (+)\n"); }
"-"     { printf("Subtraction Operator (-)\n"); }
"*"     { printf("Multiplication Operator (*)\n"); }
"/"     { printf("Division Operator (/)\n"); }

[ \t\n]+   ;       /* Ignore spaces, tabs, newlines */

.       { printf("Invalid Character: %s\n", yytext); }

%%

int yywrap()
{
   return 1;
}
```

```
int main()
{
   printf("Enter an expression: ");
   yylex();
   return 0;
}
```

Output:



5. Design a lexical Analyzer to find the number of whitespaces and newline characters.

Aim:

To design and implement a lexical analyzer using **Flex** to count the number of white spaces and newline characters present in the given input.

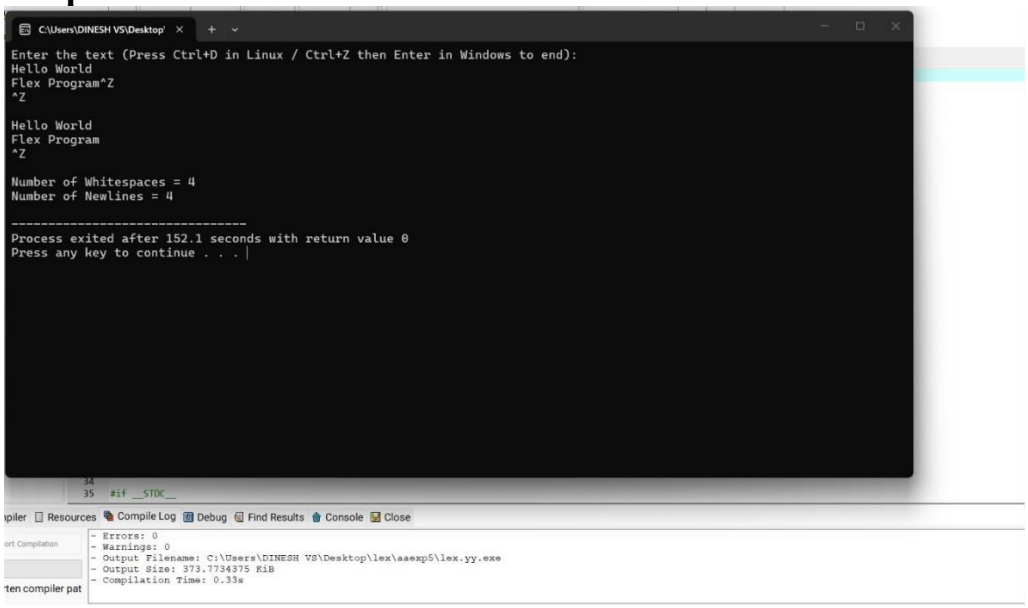
Short Procedure:

1. Write a Flex program (whitespace.l) with rules to identify white spaces and newline characters.
2. Save the program and generate the lexical analyzer using the **Flex** command.
3. Compile the generated C file using **GCC**.
4. Execute the program and enter the input text.
5. The lexical analyzer counts the number of white spaces and newline characters.
6. Display the total count of white spaces and newline characters as the output.

Code:

```
%{
#include <stdio.h>

int spaces = 0;
int newlines = 0;
%}

%%

" "         { spaces++; }
"\t"        { spaces++; }
"\n"        { newlines++; }

.           ;      /* Ignore other characters */

%%

int yywrap()
{
   return 1;
}

int main()
{
   printf("Enter the text (Press Ctrl+D in Linux / Ctrl+Z then Enter in Windows to end):\n");
   yylex();

   printf("\nNumber of Whitespaces = %d\n", spaces);
   printf("Number of Newlines = %d\n", newlines);

   return 0;
}
```

Output:



6. Develop a lexical Analyzer to test whether a given identifier is valid or not.

Aim:

To develop and implement a lexical analyzer using **Flex** to test whether a given identifier is valid according to the rules of the C programming language.

Short Procedure:

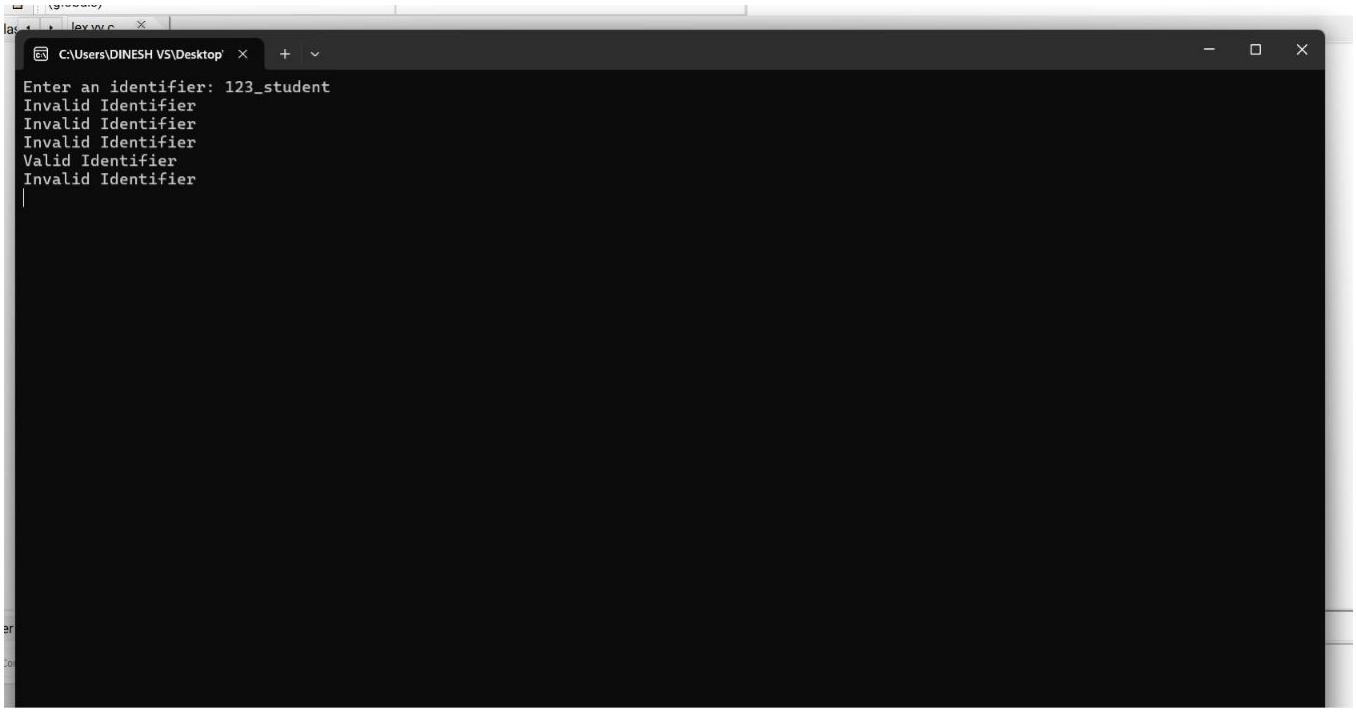
1. Write a Flex program (identifier.l) with a regular expression to recognize valid identifiers.
2. Save the program and generate the lexical analyzer using the **Flex** command.
3. Compile the generated C file using **GCC**.
4. Execute the program and enter an identifier as input.
5. The lexical analyzer checks whether the input follows the rules for a valid identifier.
6. Display **"Valid Identifier"** if the input is valid; otherwise display **"Invalid Identifier"**.

Code:

```
%{
#include <stdio.h>
%}

%%

[a-zA-Z_][a-zA-Z0-9_]*    { printf("Valid Identifier\n"); }

.|\n                      { printf("Invalid Identifier\n"); }
```

```
%%

int yywrap()
{
   return 1;
}

int main()
{
   printf("Enter an identifier: ");
   yylex();
   return 0;
}
```

Output:



7. Write a C program to find FIRST( ) - predictive parser for the given grammar
   S → AaAb / BbBa
   A → ∈
   B → ∈

Aim:

To write a C program to find the **FIRST( )** set for the given grammar using the predictive parser approach.

Grammar:

S → AaAb | BbBa
A → ε
B → ε

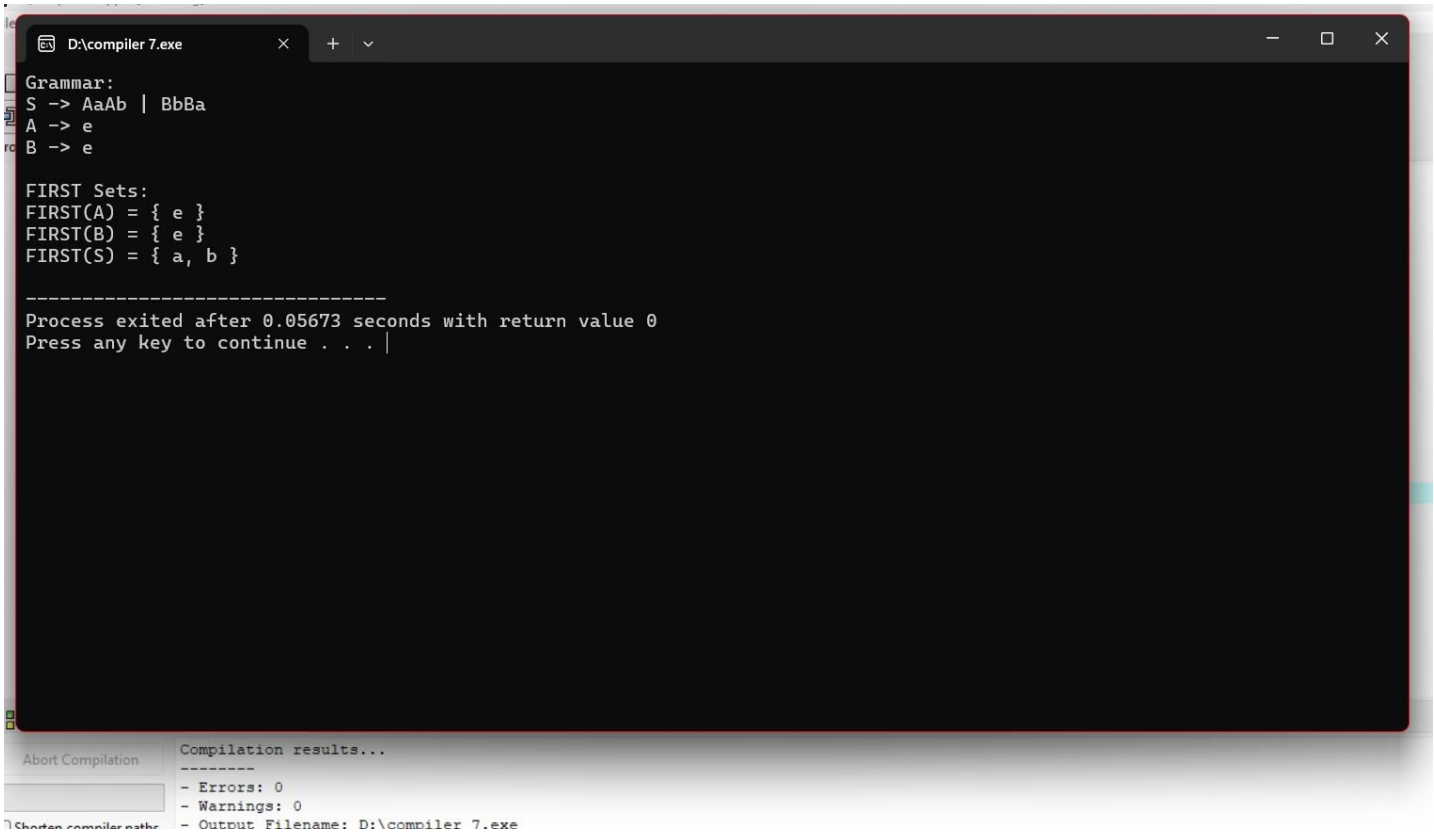
Short Procedure:

1. Define the given grammar in the C program.
2. Compute the **FIRST** set for each non-terminal.
3. If a production derives **ε (epsilon)**, include ε in the FIRST set.
4. Display the FIRST sets of all non-terminals.
5. Verify the output for the given grammar.

Code:

```
#include <stdio.h>

int main()
{
   printf("Grammar:\n");
   printf("S -> AaAb | BbBa\n");
   printf("A -> ε\n");
   printf("B -> ε\n\n");

   printf("FIRST Sets:\n");
   printf("FIRST(A) = { ε }\n");
   printf("FIRST(B) = { ε }\n");
   printf("FIRST(S) = { a, b }\n");

   return 0;
}
```

Output:



8. Write a C program to find FOLLOW( ) - predictive parser for the given grammar
   S → AaAb / BbBa
   A → ∈
   B → ∈

Aim:

To write a C program to find the **FOLLOW( )** set for the given grammar using the predictive parser approach.

Grammar:

S → AaAb | BbBa
A → ε
B → ε

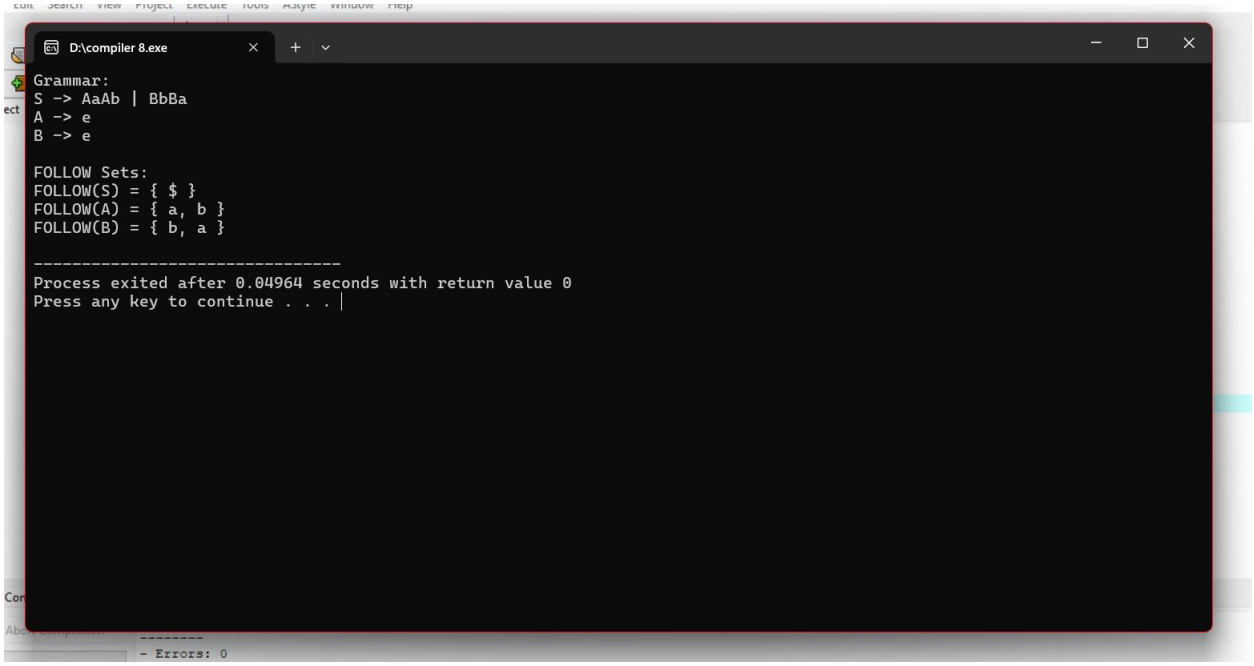
Short Procedure:

1. Define the given grammar in the C program.
2. Initialize the FOLLOW set of the start symbol with **$**.
3. Apply the FOLLOW set rules to compute the FOLLOW sets of all non-terminals.
4. Display the FOLLOW sets.
5. Verify the output for the given grammar.

Code:

```c
#include <stdio.h>

int main()
{
    printf("Grammar:\n");
    printf("S -> AaAb | BbBa\n");
    printf("A -> ε\n");
    printf("B -> ε\n\n");

    printf("FOLLOW Sets:\n");
    printf("FOLLOW(S) = { $ }\n");
    printf("FOLLOW(A) = { a, b }\n");
    printf("FOLLOW(B) = { b, a }\n");

    return 0;
}
```

Output:



9. Implement a C program to eliminate left recursion from a given CFG.

S → (L) / a
L → L , S / S

Aim:

To implement a C program to eliminate **left recursion** from the given Context-Free Grammar (CFG).

Given Grammar:

S → (L) | a
L → L,S | S

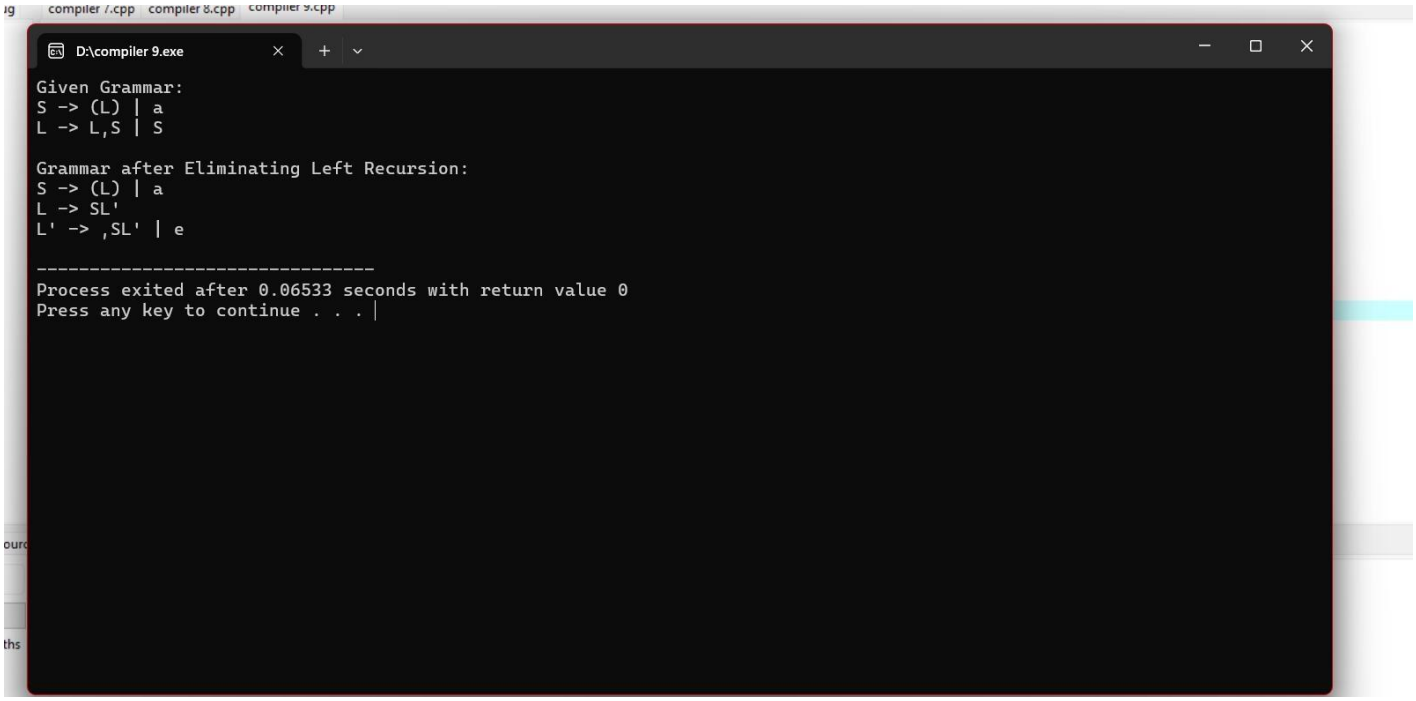
Short Procedure:

1. Define the given CFG in the C program.
2. Check the productions for immediate left recursion.
3. Apply the left recursion elimination algorithm.
4. Generate a new non-terminal to remove left recursion.
5. Display the transformed grammar without left recursion.

Code:

```
#include <stdio.h>

int main()
{
   printf("Given Grammar:\n");
   printf("S -> (L) | a\n");
   printf("L -> L,S | S\n\n");

   printf("Grammar after Eliminating Left Recursion:\n");
   printf("S -> (L) | a\n");
   printf("L -> SL'\n");
   printf("L' -> ,SL' | e\n");

   return 0;
}
```

Output:



10. Implement a C program to eliminate left factoring from a given CFG.
    S → iEtS / iEtSeS / a
    E → b

Aim:

To implement a C program to eliminate left factoring from the given Context-Free Grammar (CFG).

Given Grammar:

S → iEtS | iEtSeS | a
E → b

Short Procedure:

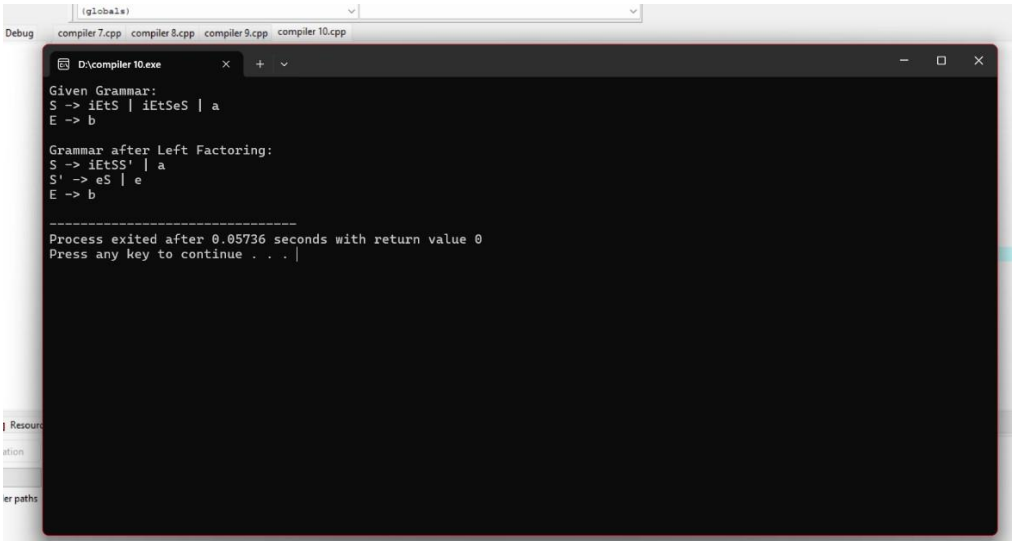
1. Define the given CFG in the C program.
2. Identify productions with a common prefix.
3. Introduce a new non-terminal to remove the common prefix.
4. Rewrite the grammar using left factoring.
5. Display the transformed grammar.

Code:

```
#include <stdio.h>
int main()
```

```
{
    printf("Given Grammar:\n");
    printf("S -> iEtS | iEtSeS | a\n");
    printf("E -> b\n\n");

    printf("Grammar after Left Factoring:\n");
    printf("S -> iEtSS' | a\n");
    printf("S' -> eS | ε\n");
    printf("E -> b\n");

    return 0;
}
```

Output:



11. Implement a C program to perform symbol table operations.

Aim:

To implement a C program to perform symbol table operations such as insertion, display, and search.

Short Procedure:

1. Define a structure for symbol table entries.
2. Read identifier details from the user.
3. Store the entries in the symbol table.
4. Display the symbol table.
5. Search for a given identifier.

Code:

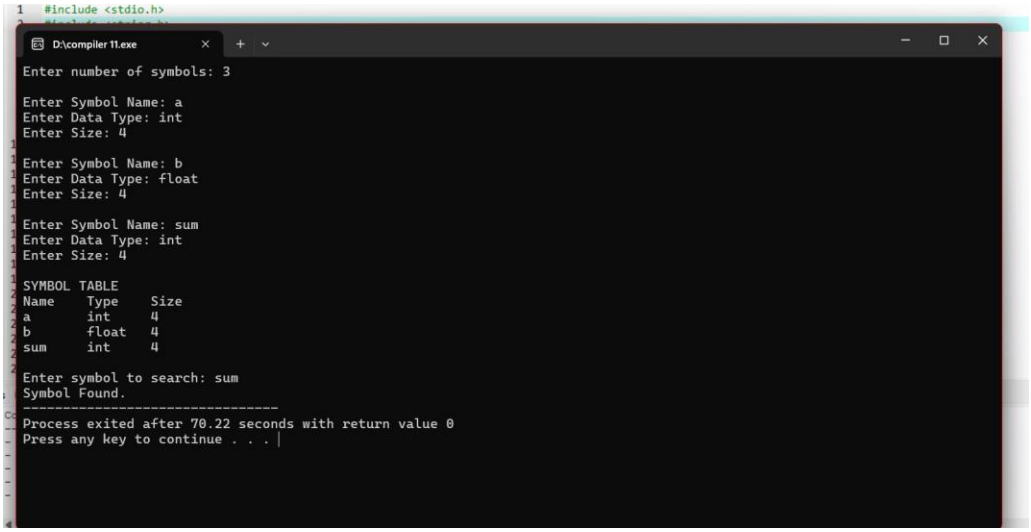
```
#include <stdio.h>
#include <string.h>

struct symbol
{
   char name[20];
   char type[20];
   int size;
};

int main()
{
   struct symbol s[20];
   int n, i, found = 0;
   char key[20];

   printf("Enter number of symbols: ");
   scanf("%d", &n);

   for(i=0;i<n;i++)
   {
      printf("\nEnter Symbol Name: ");
      scanf("%s", s[i].name);

      printf("Enter Data Type: ");
      scanf("%s", s[i].type);

      printf("Enter Size: ");
      scanf("%d", &s[i].size);
   }

   printf("\nSYMBOL TABLE\n");
   printf("Name\tType\tSize\n");

   for(i=0;i<n;i++)
      printf("%s\t%s\t%d\n", s[i].name, s[i].type, s[i].size);

   printf("\nEnter symbol to search: ");
   scanf("%s", key);

   for(i=0;i<n;i++)
   {
```

```
        if(strcmp(key,s[i].name)==0)
        {
            found=1;
            printf("Symbol Found.");
            break;
        }
    }

    if(!found)
        printf("Symbol Not Found.");

    return 0;
}
```

Sample Input:

```
3
a
int
4
b
float
4
sum
int
4
sum
```

Output:



12. Write a C program to construct recursive descent parsing for the given grammar

E → TE'
E' → +TE' / ∈
T → FT'
T' → *FT' / ∈
F → (E) / id

Aim:

To implement Recursive Descent Parsing for the given grammar.

Grammar:

E → TE'
E' → +TE' | ε
T → FT'
T' → *FT' | ε
F → (E) | id

Short Procedure:

1. Read the input expression.
2. Implement functions E(), E1(), T(), T1(), and F().
3. Parse the input recursively.
4. Accept if the entire string matches the grammar.

Code:

```
#include<stdio.h>
#include<string.h>

char str[100];
int i=0;

void E();
void E1();
void T();
void T1();
void F();

void E()
{
   T();
   E1();
```

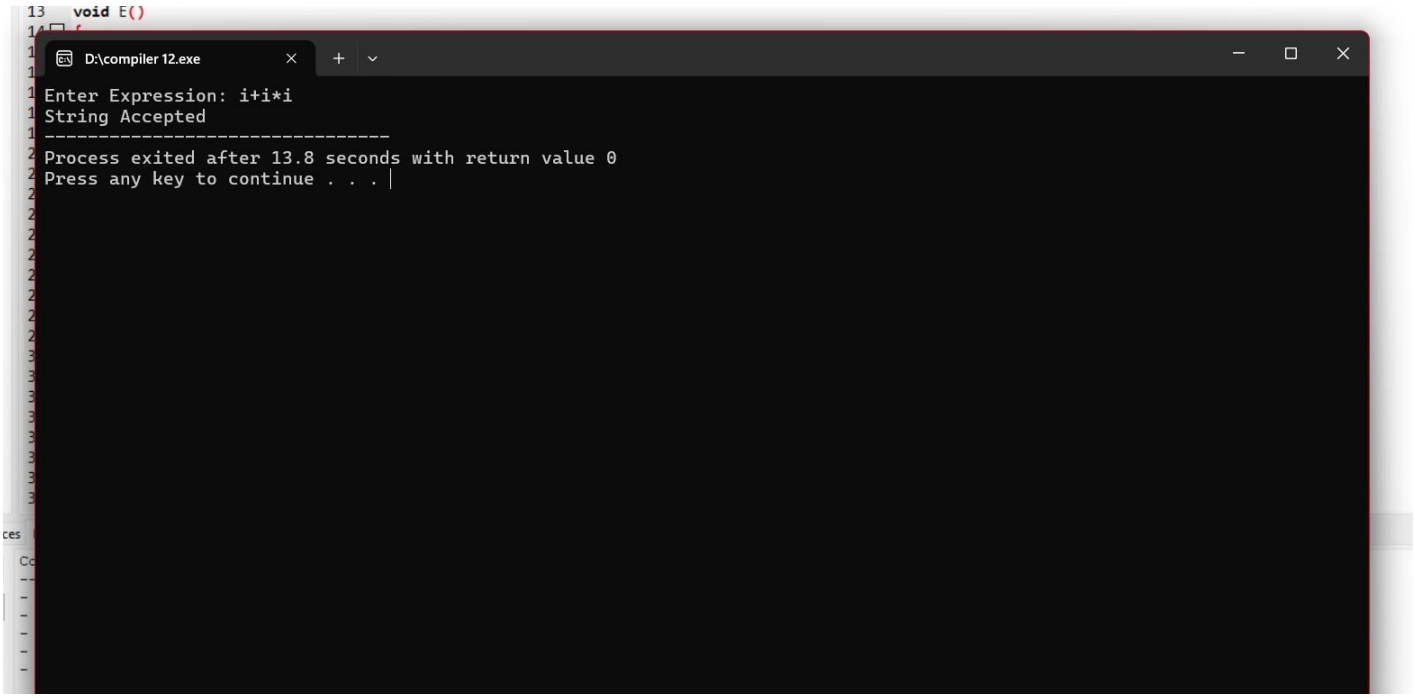
```
}

void E1()
{
   if(str[i]=='+')
   {
      i++;
      T();
      E1();
   }
}

void T()
{
   F();
   T1();
}

void T1()
{
   if(str[i]=='*')
   {
      i++;
      F();
      T1();
   }
}

void F()
{
   if(str[i]=='i')
      i++;
   else if(str[i]=='(')
   {
      i++;
      E();
      if(str[i]==')')
         i++;
   }
}

int main()
{
   printf("Enter Expression: ");
```

```
    scanf("%s",str);

    E();

    if(str[i]=='\0')
        printf("String Accepted");
    else
        printf("String Rejected");

    return 0;
}
```

Sample Input:

i+i*i

Output:



13. **All languages have Grammar. When people frame a sentence we usually say whether the sentence is framed as per the rules of the Grammar or Not. Similarly use the same ideology, implement either Top Down parsing technique or Bottom Up Parsing technique to check whether the given input string is satisfying the grammar or not.**

Aim:

To implement a Top Down Parser to check whether the given input string satisfies the grammar.

Short Procedure:

1. Read the input string.
2. Parse using recursive functions.
3. Compare with grammar rules.
4. Display Accepted or Rejected.

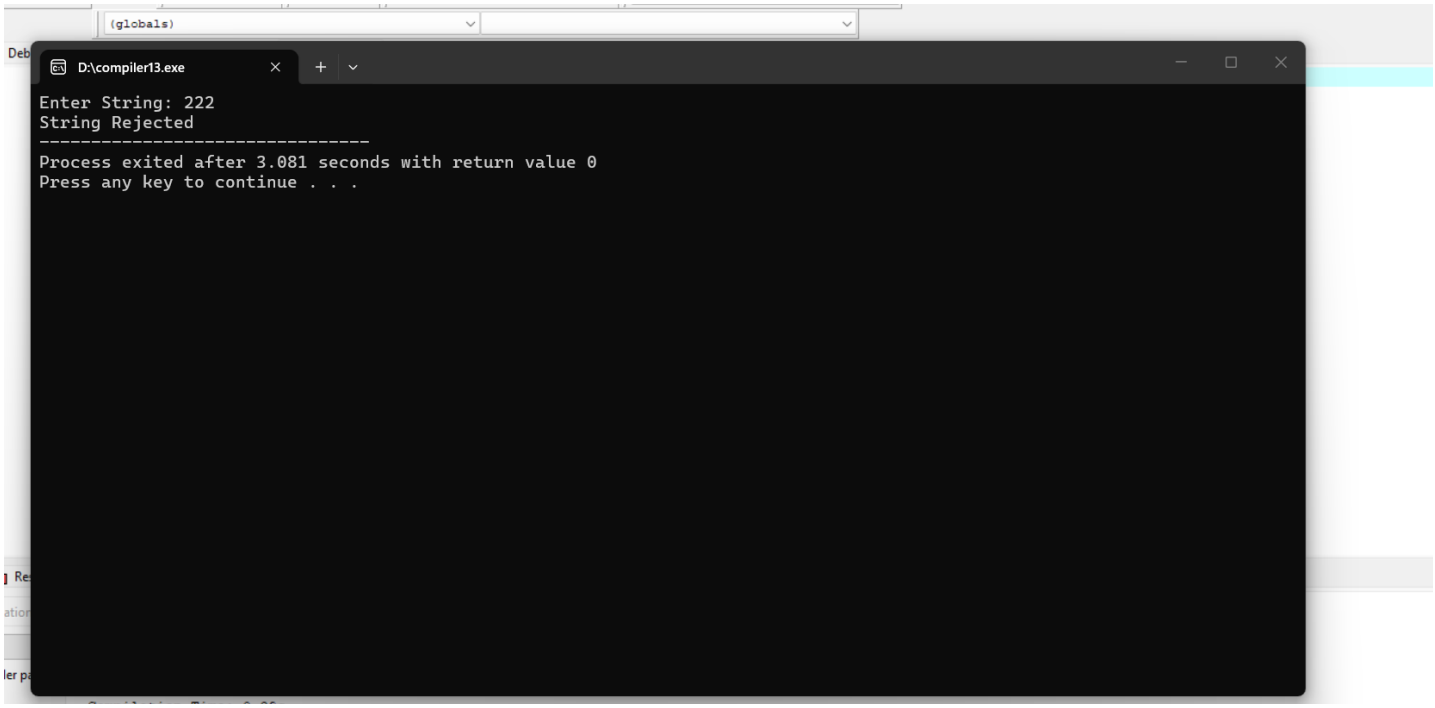
Code:

```
#include<stdio.h>
#include<string.h>

char input[20];
int pos=0;

void S()
{
   if(input[pos]=='a')
      pos++;
}

int main()
{
   printf("Enter String: ");
   scanf("%s",input);

   S();

   if(pos==strlen(input))
      printf("String Accepted");
   else
      printf("String Rejected");

   return 0;
}
```

Sample Input:

a

Output:



14. **The main function of the Intermediate code generation is producing three address code statements for a given input expression. The three address codes help in determining the sequence in which operations are actioned by the compiler. The key work of Intermediate code generators is to simplify the process of Code generators. Write a C Program to Generate the Three address code representation for the given input statement.**

Aim:

To generate Three Address Code (TAC) for a given arithmetic expression.

Short Procedure:

1. Read the arithmetic expression.
2. Scan operators from left to right.
3. Generate temporary variables.
4. Print the Three Address Code.

Code:

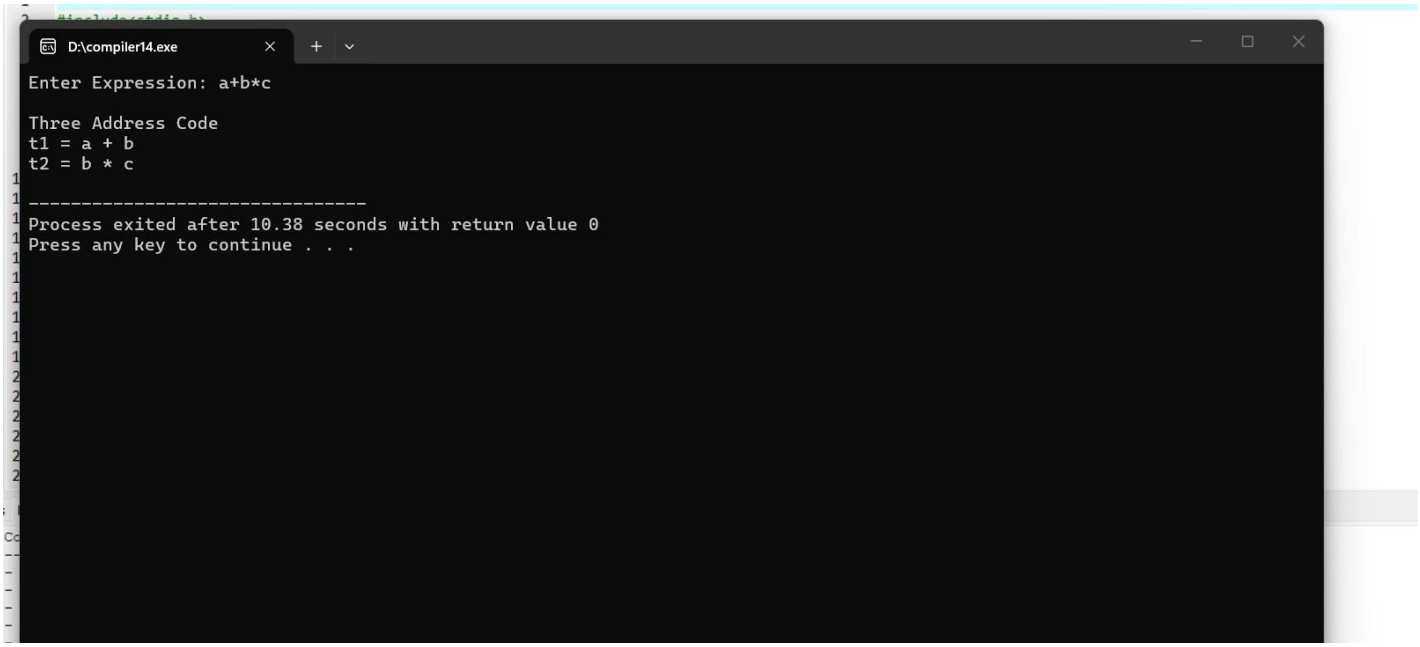
```
#include<stdio.h>
#include<string.h>
```

```
int main()
{
   char exp[20];
   int i,temp=1;

   printf("Enter Expression: ");
   scanf("%s",exp);

   printf("\nThree Address Code\n");

   for(i=0; exp[i]!='\0'; i++)
   {
      if(exp[i]=='+'||exp[i]=='-'||exp[i]=='*'||exp[i]=='/')
      {
         printf("t%d = %c %c %c\n",temp,exp[i-1],exp[i],exp[i+1]);
         temp++;
      }
   }

   return 0;
}
```

Sample Input:

a+b*c

Output:



15. Write a C program for implementing a Lexical Analyzer to Scan and Count the number of characters, words, and lines in a file.

Aim:

To implement a lexical analyzer that scans a file and counts the number of characters, words, and lines.

Short Procedure:

1. Open the input file.
2. Read characters one by one.
3. Count characters, words, and lines.
4. Display the results.

Code:

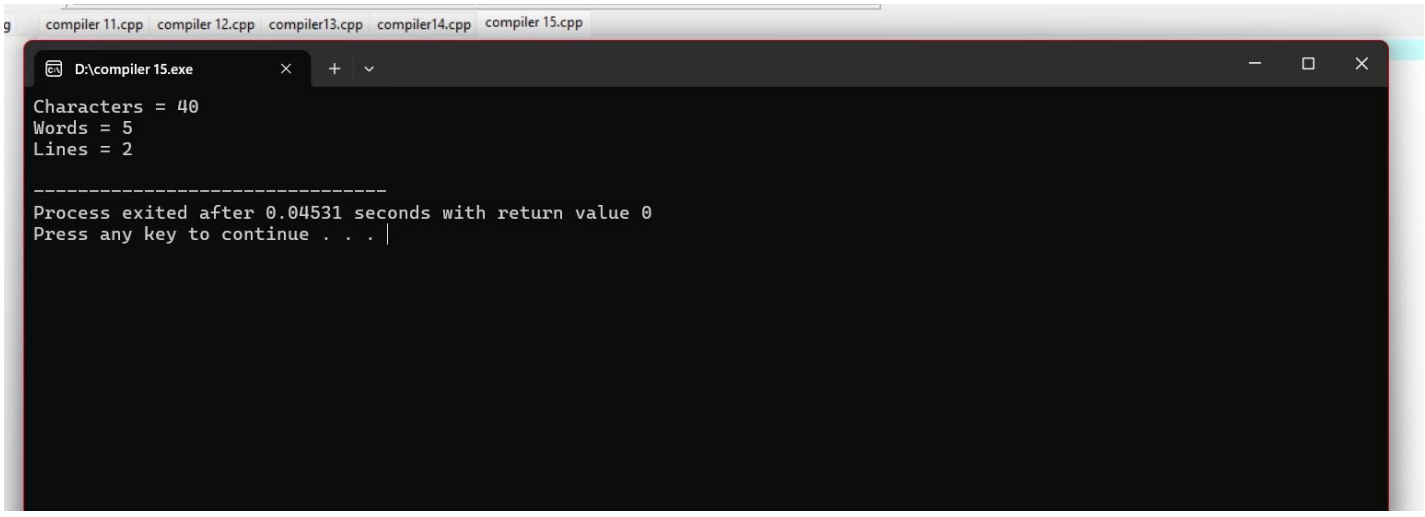
```
#include<stdio.h>
#include<stdlib.h>
#include<ctype.h>

int main()
{
   FILE *fp;
   char ch;
   int chars=0,words=0,lines=0,inWord=0;

   fp=fopen("sample.txt","r");

   if(fp==NULL)
   {
      printf("File not found");
      return 0;
   }

   while((ch=fgetc(fp))!=EOF)
   {
      chars++;

      if(ch=='\n')
         lines++;

      if(isspace(ch))
         inWord=0;
```

```
        else if(inWord==0)
        {
            inWord=1;
            words++;
        }
    }

    fclose(fp);

    printf("Characters = %d\n",chars);
    printf("Words = %d\n",words);
    printf("Lines = %d\n",lines+1);

    return 0;
}
```

Sample Input (sample.txt):

Compiler Design Lab
Lexical Analyzer Program

Output:



16. Write a C program to implement the back end of the compiler.

Aim:

To implement the back end of a compiler for simple target code generation.

Short Procedure:

1. Read a simple assignment expression.

2. Identify the operands and operator.
3. Generate simple assembly-like instructions.
4. Display the target code.

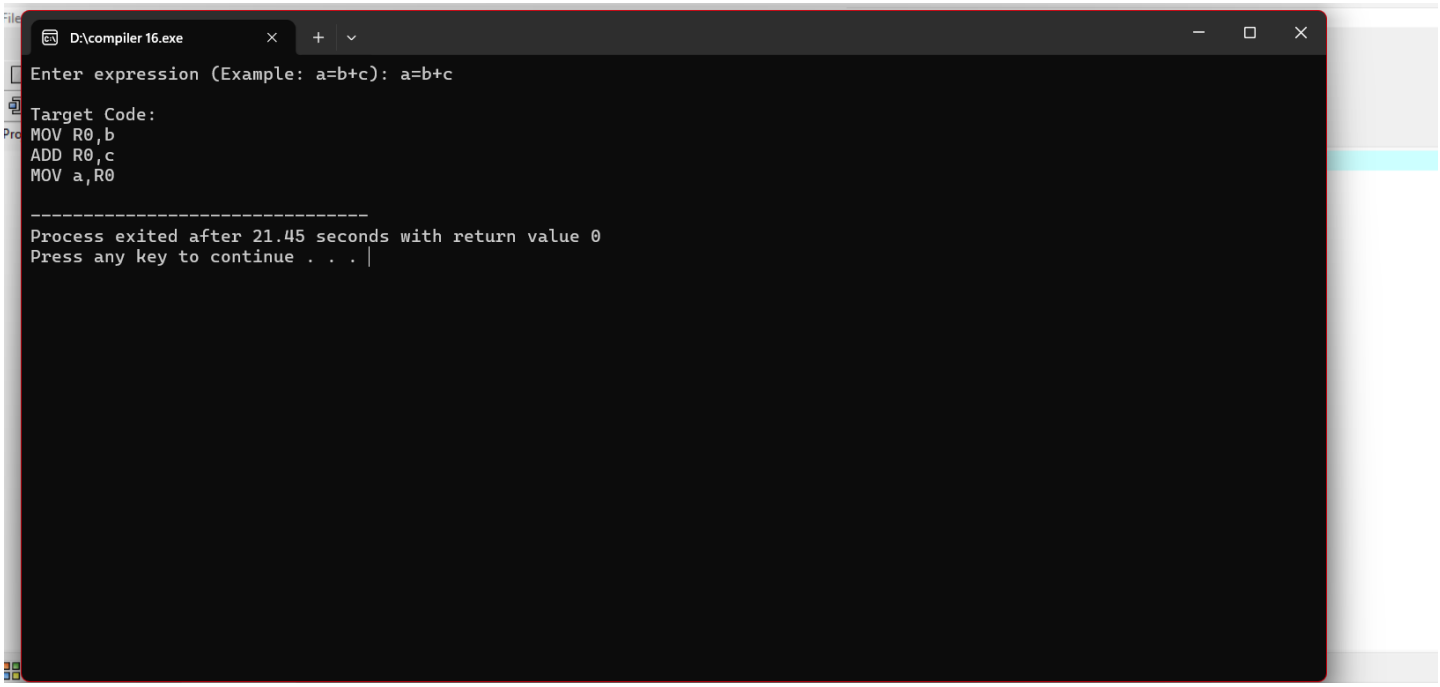
Code:

```
#include <stdio.h>

int main()
{
   char a, b, c, op;

   printf("Enter expression (Example: a=b+c): ");
   scanf("%c=%c%c%c", &a, &b, &op, &c);

   printf("\nTarget Code:\n");
   printf("MOV R0,%c\n", b);

   switch(op)
   {
      case '+':
         printf("ADD R0,%c\n", c);
         break;
      case '-':
         printf("SUB R0,%c\n", c);
         break;
      case '*':
         printf("MUL R0,%c\n", c);
         break;
      case '/':
         printf("DIV R0,%c\n", c);
         break;
   }

   printf("MOV %c,R0\n", a);

   return 0;
}
```

Sample Input:

```
a=b+c
```

Output:



17. Write a C program to compute LEADING( ) – operator precedence parser for the given grammar

E → E + T | T
T → T * F | F
F → (E) | id

Aim:

To write a C program to compute the LEADING() set for the given operator precedence grammar.

Short Procedure:

1. Define the grammar productions.
2. Read the productions into the program.
3. Apply the LEADING() computation rules.
4. Store the terminals in the LEADING set.
5. Display the LEADING set for each non-terminal.

Code:

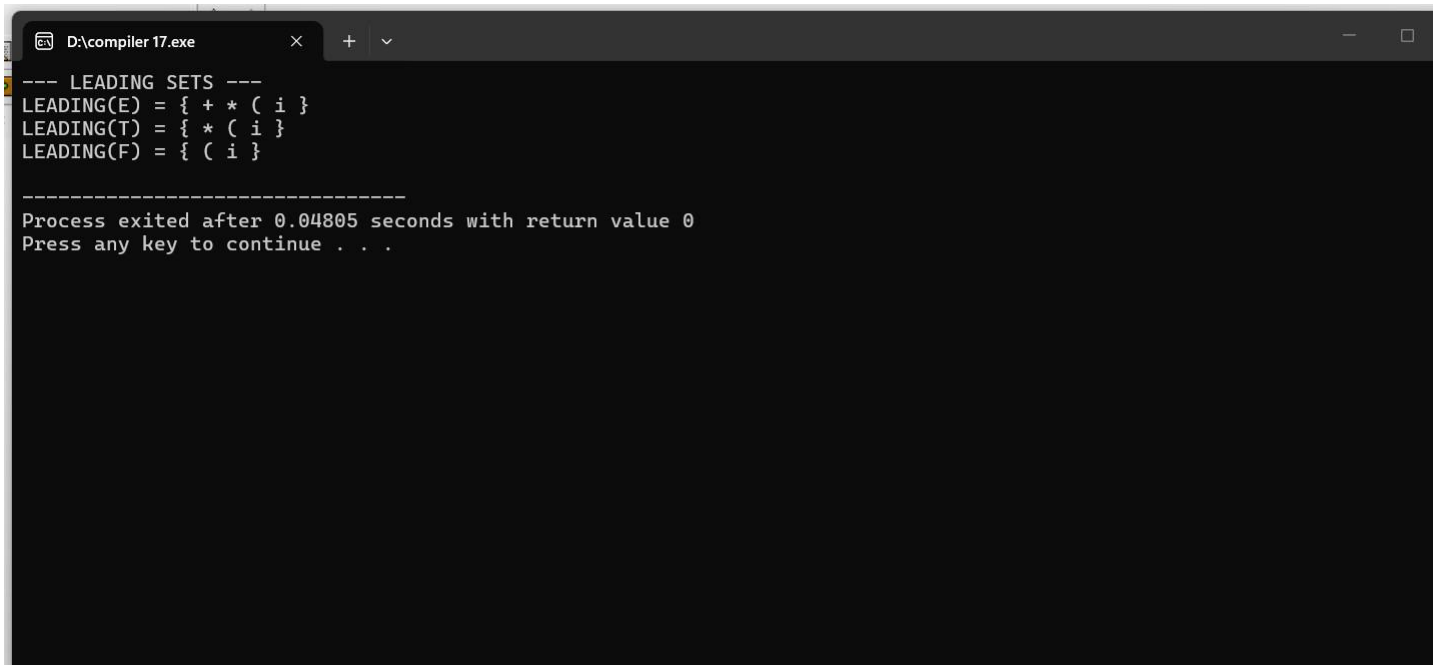
```
#include <stdio.h>
#include <string.h>

char production[10][10];
```

```
int n;
char non_terminals[10];
int nt_count = 0;
char leading[10][10];

int is_non_terminal(char c) {
    return (c >= 'A' && c <= 'Z');
}

void add_leading(int nt_idx, char symbol) {
    for (int i = 0; i < strlen(leading[nt_idx]); i++) {
        if (leading[nt_idx][i] == symbol) return;
    }
    int len = strlen(leading[nt_idx]);
    leading[nt_idx][len] = symbol;
    leading[nt_idx][len + 1] = '\0';
}

void compute_leading() {
    int changed;
    do {
        changed = 0;
        for (int i = 0; i < n; i++) {
            char lhs = production[i][0];
            int lhs_idx = -1;
            for (int k = 0; k < nt_count; k++) {
                if (non_terminals[k] == lhs) { lhs_idx = k; break; }
            }

            char *rhs = production[i] + 3; // skip "X->"

            // Case 1: First symbol is terminal
            if (!is_non_terminal(rhs[0])) {
                int prev_len = strlen(leading[lhs_idx]);
                add_leading(lhs_idx, rhs[0]);
                if (strlen(leading[lhs_idx]) > prev_len) changed = 1;
            }
            // Case 2: First symbol is non-terminal, second is terminal (e.g. E -> E + T)
            else {
                if (rhs[1] != '\0' && !is_non_terminal(rhs[1])) {
                    int prev_len = strlen(leading[lhs_idx]);
                    add_leading(lhs_idx, rhs[1]);
                    if (strlen(leading[lhs_idx]) > prev_len) changed = 1;
                }
```

```
            // Case 3: Inherit LEADING from leading non-terminal
            int rhs_nt_idx = -1;
            for (int k = 0; k < nt_count; k++) {
                if (non_terminals[k] == rhs[0]) { rhs_nt_idx = k; break; }
            }
            if (rhs_nt_idx != -1) {
                for (int j = 0; j < strlen(leading[rhs_nt_idx]); j++) {
                    int prev_len = strlen(leading[lhs_idx]);
                    add_leading(lhs_idx, leading[rhs_nt_idx][j]);
                    if (strlen(leading[lhs_idx]) > prev_len) changed = 1;
                }
            }
        }
    }
  } while (changed);
}

int main() {
  n = 6;
  strcpy(production[0], "E->E+T");
  strcpy(production[1], "E->T");
  strcpy(production[2], "T->T*F");
  strcpy(production[3], "T->F");
  strcpy(production[4], "F->(E)");
  strcpy(production[5], "F->i"); // 'i' represents 'id'

  non_terminals[0] = 'E';
  non_terminals[1] = 'T';
  non_terminals[2] = 'F';
  nt_count = 3;

  for (int i = 0; i < nt_count; i++) leading[i][0] = '\0';

  compute_leading();

  printf("--- LEADING SETS ---\n");
  for (int i = 0; i < nt_count; i++) {
    printf("LEADING(%c) = { ", non_terminals[i]);
    for (int j = 0; j < strlen(leading[i]); j++) {
      printf("%c ", leading[i][j]);
    }
    printf("}\n");
  }
  return 0;
```

}

Output:



18. Write a C program to compute TRAILING( ) – operator precedence parser for the given grammar
E → E + T | T
T → T * F | F
F → (E) | id

Aim:

To write a C program to compute the TRAILING() set for the given operator precedence grammar.

Short Procedure:

1. Define the grammar productions.
2. Read the grammar into the program.
3. Apply the TRAILING() computation rules.
4. Store the terminals in the TRAILING set.
5. Display the TRAILING set for each non-terminal**.**

Code:

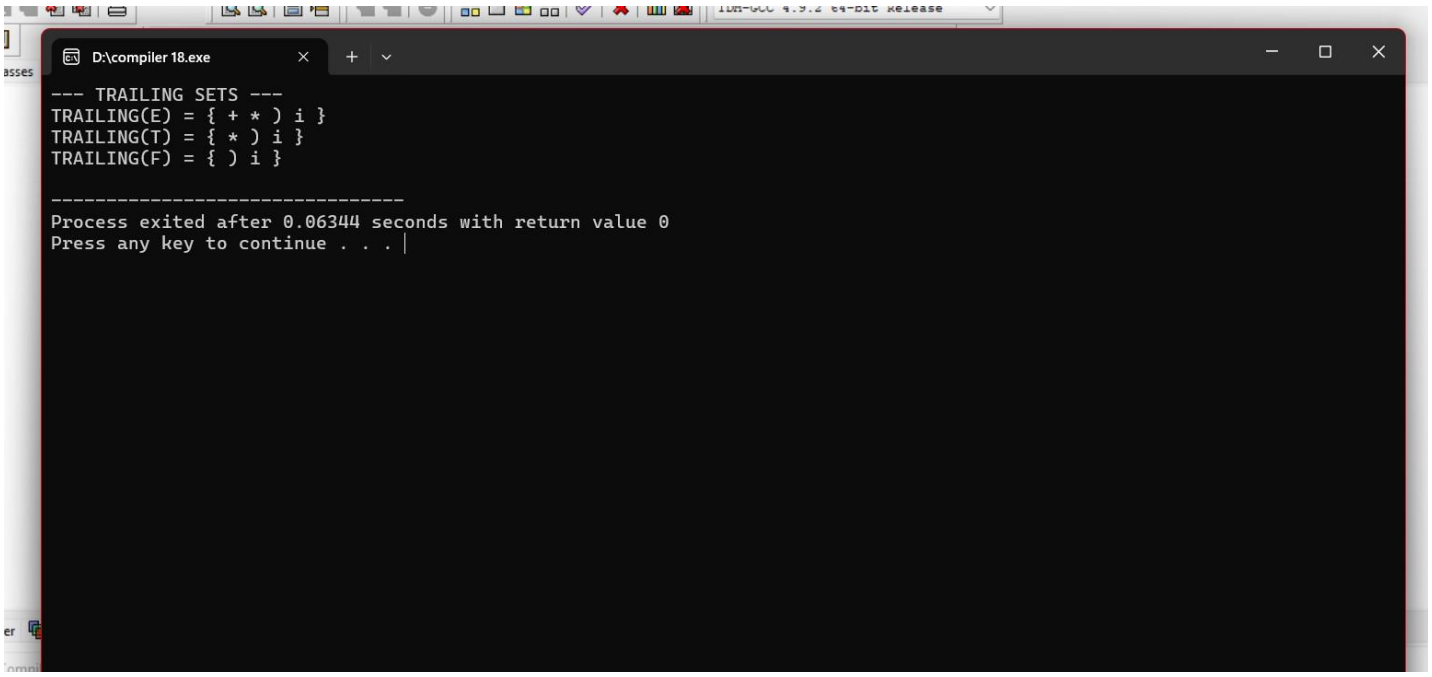
#include <stdio.h>

```
#include <string.h>

char production[10][10];
int n;
char non_terminals[10];
int nt_count = 0;
char trailing[10][10];

int is_non_terminal(char c) {
    return (c >= 'A' && c <= 'Z');
}

void add_trailing(int nt_idx, char symbol) {
    for (int i = 0; i < strlen(trailing[nt_idx]); i++) {
        if (trailing[nt_idx][i] == symbol) return;
    }
    int len = strlen(trailing[nt_idx]);
    trailing[nt_idx][len] = symbol;
    trailing[nt_idx][len + 1] = '\0';
}

void compute_trailing() {
    int changed;
    do {
        changed = 0;
        for (int i = 0; i < n; i++) {
            char lhs = production[i][0];
            int lhs_idx = -1;
            for (int k = 0; k < nt_count; k++) {
                if (non_terminals[k] == lhs) { lhs_idx = k; break; }
            }

            char *rhs = production[i] + 3;
            int rlen = strlen(rhs);

            // Case 1: Last symbol is terminal
            if (!is_non_terminal(rhs[rlen - 1])) {
                int prev_len = strlen(trailing[lhs_idx]);
                add_trailing(lhs_idx, rhs[rlen - 1]);
                if (strlen(trailing[lhs_idx]) > prev_len) changed = 1;
            }
            // Case 2: Last symbol is non-terminal, second last is terminal
            else {
                if (rlen >= 2 && !is_non_terminal(rhs[rlen - 2])) {
```

```
                int prev_len = strlen(trailing[lhs_idx]);
                add_trailing(lhs_idx, rhs[rlen - 2]);
                if (strlen(trailing[lhs_idx]) > prev_len) changed = 1;
            }
            // Case 3: Inherit TRAILING from trailing non-terminal
            int rhs_nt_idx = -1;
            for (int k = 0; k < nt_count; k++) {
                if (non_terminals[k] == rhs[rlen - 1]) { rhs_nt_idx = k; break; }
            }
            if (rhs_nt_idx != -1) {
                for (int j = 0; j < strlen(trailing[rhs_nt_idx]); j++) {
                    int prev_len = strlen(trailing[lhs_idx]);
                    add_trailing(lhs_idx, trailing[rhs_nt_idx][j]);
                    if (strlen(trailing[lhs_idx]) > prev_len) changed = 1;
                }
            }
        }
    }
  } while (changed);
}

int main() {
  n = 6;
  strcpy(production[0], "E->E+T");
  strcpy(production[1], "E->T");
  strcpy(production[2], "T->T*F");
  strcpy(production[3], "T->F");
  strcpy(production[4], "F->(E)");
  strcpy(production[5], "F->i");

  non_terminals[0] = 'E';
  non_terminals[1] = 'T';
  non_terminals[2] = 'F';
  nt_count = 3;

  for (int i = 0; i < nt_count; i++) trailing[i][0] = '\0';

  compute_trailing();

  printf("--- TRAILING SETS ---\n");
  for (int i = 0; i < nt_count; i++) {
    printf("TRAILING(%c) = { ", non_terminals[i]);
    for (int j = 0; j < strlen(trailing[i]); j++) {
      printf("%c ", trailing[i][j]);
```

```
        }
        printf("}\n");
    }
    return 0;
}
```

Output:



19. **The lexical analyzer should ignore redundant spaces, tabs and new lines. It should also ignore comments. Although the syntax specification states that identifiers can be arbitrarily long, you may restrict the length to some reasonable value. Write a LEX specification file to take input C program from a .c file and count the number of characters, number of lines & number of words.**

    Input Source Program: (sample.c)

```
#include <stdio.h>
int main()
{
int number1, number2, sum;
printf("Enter two integers: ");
scanf("%d %d", &number1, &number2);
sum = number1 + number2;
printf("%d + %d = %d", number1, number2, sum);
return 0;
}
```

Aim:

To write a LEX program that ignores spaces, tabs, new lines and comments while counting characters, words and lines in a C source file.

Short Procedure:

1. Create a LEX specification file.
2. Define patterns for comments and white spaces.
3. Ignore comments and extra white spaces.
4. Count characters, words and lines.
5. Display the final count.

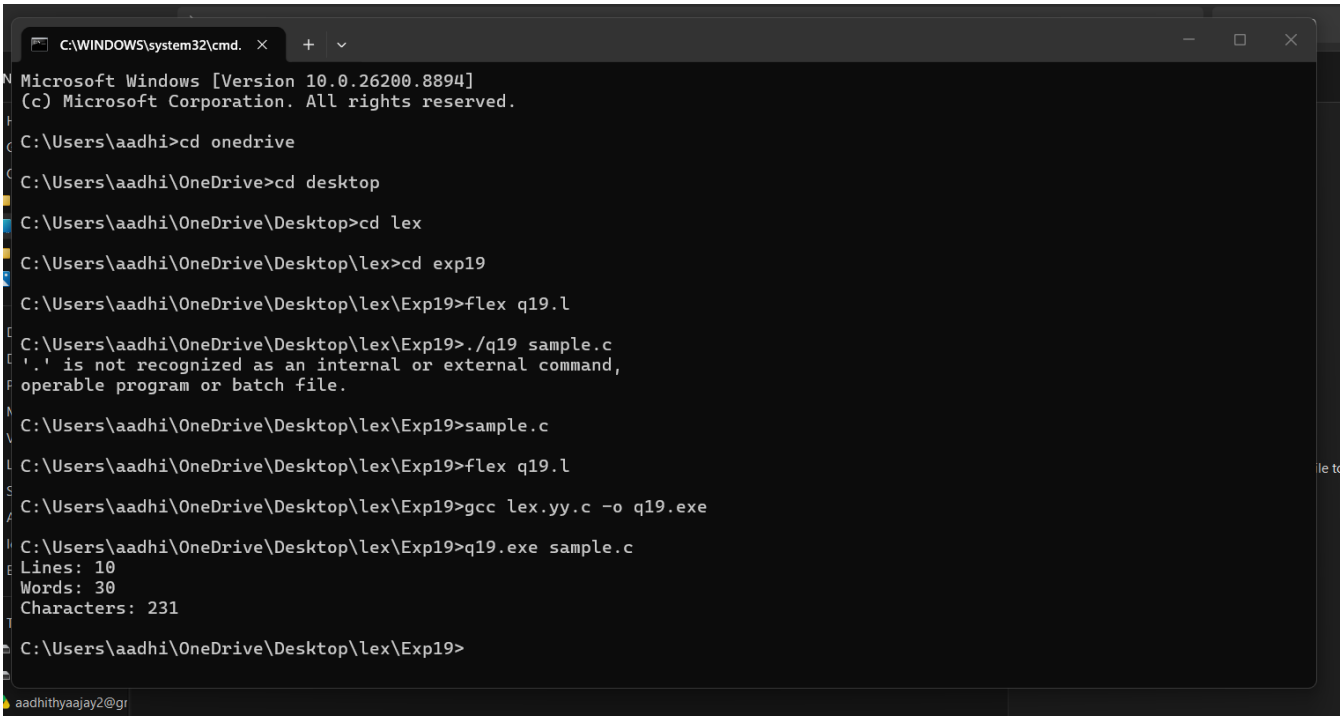
Code:

```
%{
#include <stdio.h>
int char_count = 0, word_count = 0, line_count = 0;
%}

%option noyywrap

%%

"/*"([^*]|\*+[^/*])*\*+"/"     { /* Ignore multi-line comments */ }
"//".*                         { /* Ignore single-line comments */ }
\n                             { line_count++; char_count++; }
[ \t]+                         { char_count += yyleng; }
[a-zA-Z0-9_]{1,31}             { word_count++; char_count += yyleng; }
.                              { char_count++; }

%%

int main(int argc, char **argv) {
   if(argc > 1) {
      FILE *fp = fopen(argv[1], "r");
      if(fp) yyin = fp;
   }
   yylex();
   printf("Lines: %d\nWords: %d\nCharacters: %d\n", line_count, word_count,
char_count);
   return 0;
}
```

Sample Input:

```
#include <stdio.h>
int main()
{
int number1, number2, sum;
printf("Enter two integers: ");
scanf("%d %d", &number1, &number2);
sum = number1 + number2;
printf("%d + %d = %d", number1, number2, sum);
return 0;
}
```

Output:



20. **Write a LEX program to print all the constants in the given C source program file.**
    Input Source Program: (sample.c)

```
#define PI 3.14
#include<stdio.h>
#include<conio.h>
void main()
{
int a,b,c = 30;
printf("hello");
}
```

Write a LEX program to count the number of Macros defined and header files included in the C program.

Input Source Program: (sample.c)

```
#define PI 3.14
#include<stdio.h>
#include<conio.h>
void main()
{
int a,b,c = 30;
printf("hello");
}
```

Aim:

To write a LEX program to identify constants and count macros and header files in a C program.

Short Procedure:

1. Define patterns for constants.
2. Define patterns for macros and header files.
3. Read the input C program.
4. Count and print matched tokens.
5. Display the total count.

Code:

Part A: All constant

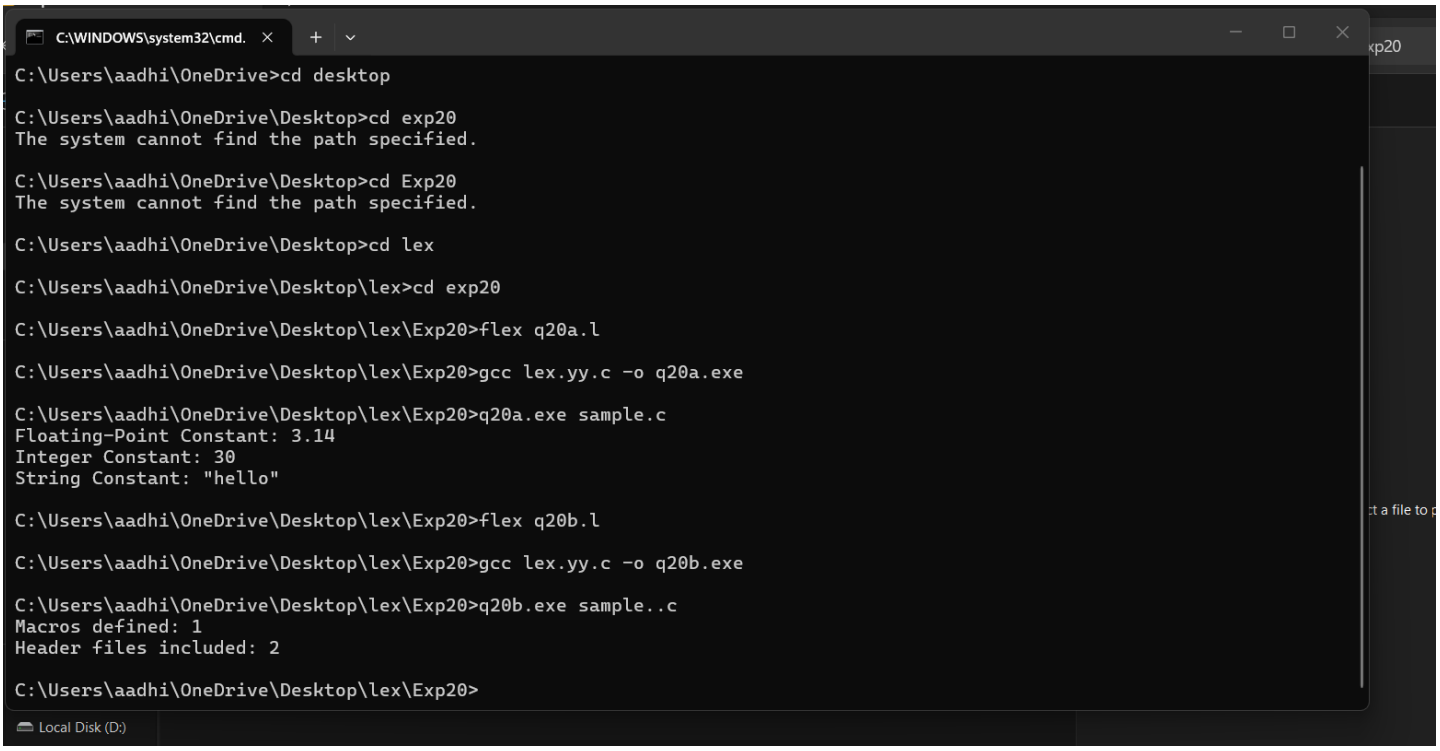
```
%{
#include <stdio.h>
%}

%option noyywrap

%%

\"[^\"]*\"              { printf("String Constant: %s\n", yytext); }
'[^']'                  { printf("Character Constant: %s\n", yytext); }
[0-9]+\.[0-9]+          { printf("Floating-Point Constant: %s\n", yytext); }
[0-9]+                  { printf("Integer Constant: %s\n", yytext); }
.|\n                    { /* Ignore other tokens */ }
```

```
%%

int main(int argc, char **argv) {
   if(argc > 1) {
      FILE *f = fopen(argv[1], "r");
      if(f) yyin = f;
   }
   yylex();
   return 0;
}
```

Part B: Count Macros and Header files

```
%{
#include <stdio.h>
int macro_count = 0;
int header_count = 0;
%}

%option noyywrap

%%

#define[ \t]+[A-Za-z_][A-Za-z0-9_]*.*    { macro_count++; }
#include[ \t]*[<"][^>"]+[>"]            { header_count++; }
.|\n                                { /* Ignore rest */ }

%%

int main(int argc, char **argv) {
   if(argc > 1) {
      FILE *f = fopen(argv[1], "r");
      if(f) yyin = f;
   }
   yylex();
   printf("Macros defined: %d\n", macro_count);
   printf("Header files included: %d\n", header_count);
   return 0;
}
```

Output:



21. **In a class, an English teacher was teaching the vowels and consonants to the students. She says "Vowel sounds allow the air to flow freely, causing the chin to drop noticeably, whilst consonant sounds are produced by restricting the air flow". As a class activity the students are asked to identify the vowels and consonants in the given word/sentence and count the number of elements in each. Write an algorithm to help the student to count the number of vowels in the given sentence.**

Aim:

To write an algorithm/program to count the number of vowels in a given sentence.

Short Procedure:

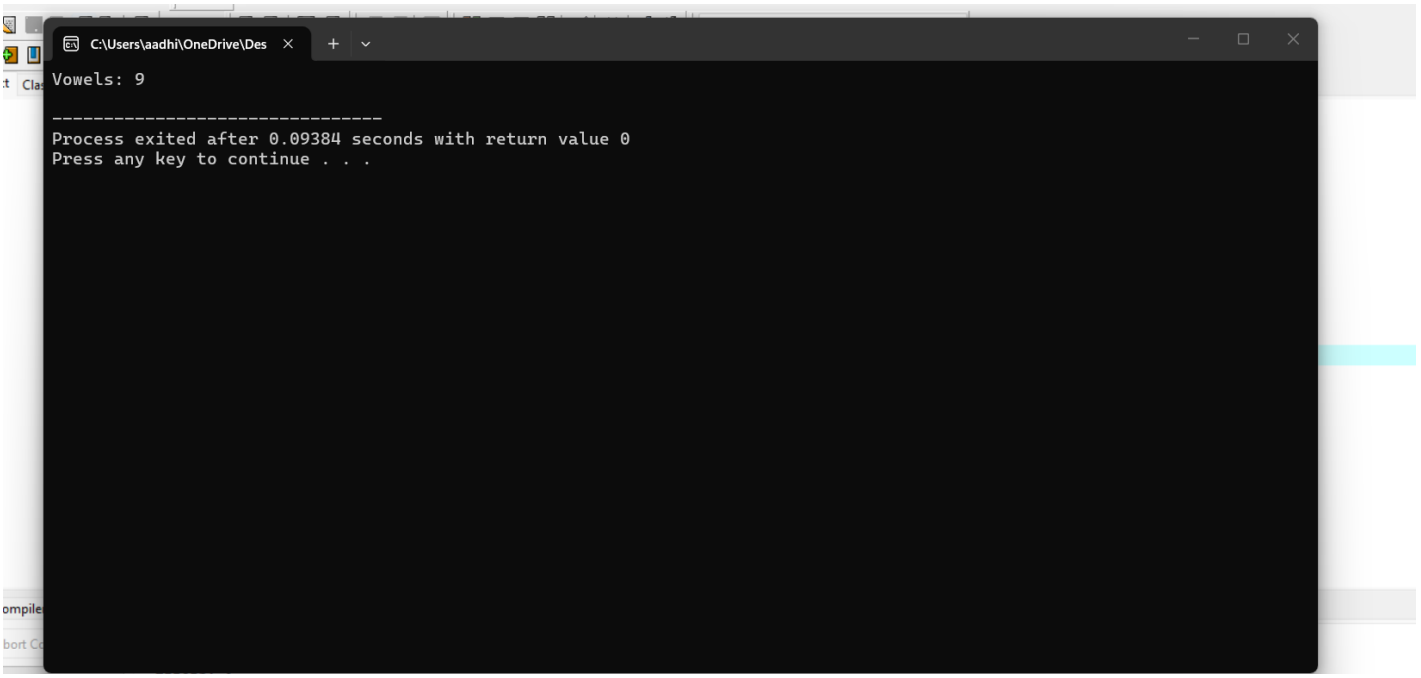
1. Read the input sentence.
2. Traverse each character.
3. Check whether it is a vowel.
4. Increment the vowel count.
5. Display the total vowels.

Code:

```
#include <stdio.h>
#include <ctype.h>
```

```
int main() {
    char str[] = "Compiler Design Laboratory";
    int count = 0;
    for(int i = 0; str[i] != '\0'; i++) {
        char ch = tolower(str[i]);
        if(ch == 'a' || ch == 'e' || ch == 'i' || ch == 'o' || ch == 'u') count++;
    }
    printf("Vowels: %d\n", count);
    return 0;
}
```

Output:



22. Write a LEX program to print all HTML tags in the input file.
Input Source Program: (sample.html)

```
<html>
<body>
<h1>My First Heading</h1>
<p>My first paragraph.</p>
</body>
</html>
```

Aim:

To write a LEX program to identify and print HTML tags from an HTML file.

Short Procedure:

1. Read the HTML file.
2. Match HTML tag patterns.
3. Print each identified tag.
4. Ignore normal text.
5. Display all HTML tags.

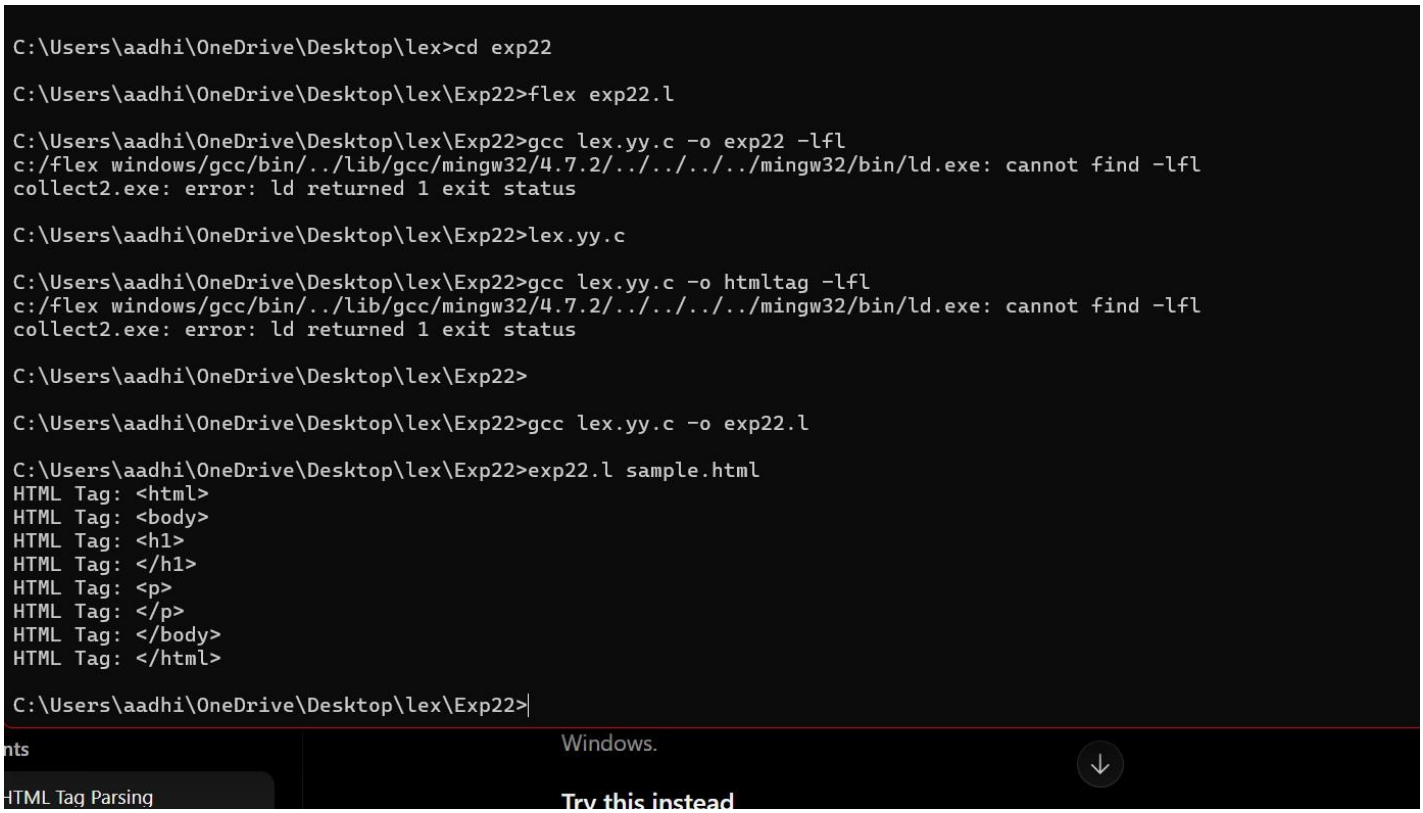
Code:

```
%{
#include <stdio.h>
%}

%option noyywrap

%%

"<"[^>]+">"    { printf("HTML Tag: %s\n", yytext); }
.|\n           { /* Ignore non-tag text */ }

%%

int main(int argc, char **argv) {
   if (argc > 1) {
      FILE *fp = fopen(argv[1], "r");
      if (fp) yyin = fp;
   }
   yylex();
   return 0;
} int yywrap()
{
return 1;
}
```

Sample input:

```
<html>
<body>
<h1>My First Heading</h1>
<p>My first paragraph.</p>
</body>
</html>
```

Output:



23. Write a LEX program which adds line numbers to the given C program file and display
the same in the standard output.
Input Source Program: (sample.c)

```
#define PI 3.14
#include<stdio.h>
#include<conio.h>
void main()
{
int a,b,c = 30;
printf("hello");
}
```

Aim:

To write a LEX program to display a C program with line numbers.

Short Procedure:

1. Read the C source file.
2. Initialize the line counter.
3. Print each line with its line number.

4. Increment the counter.
5. Display the numbered program.

Code:

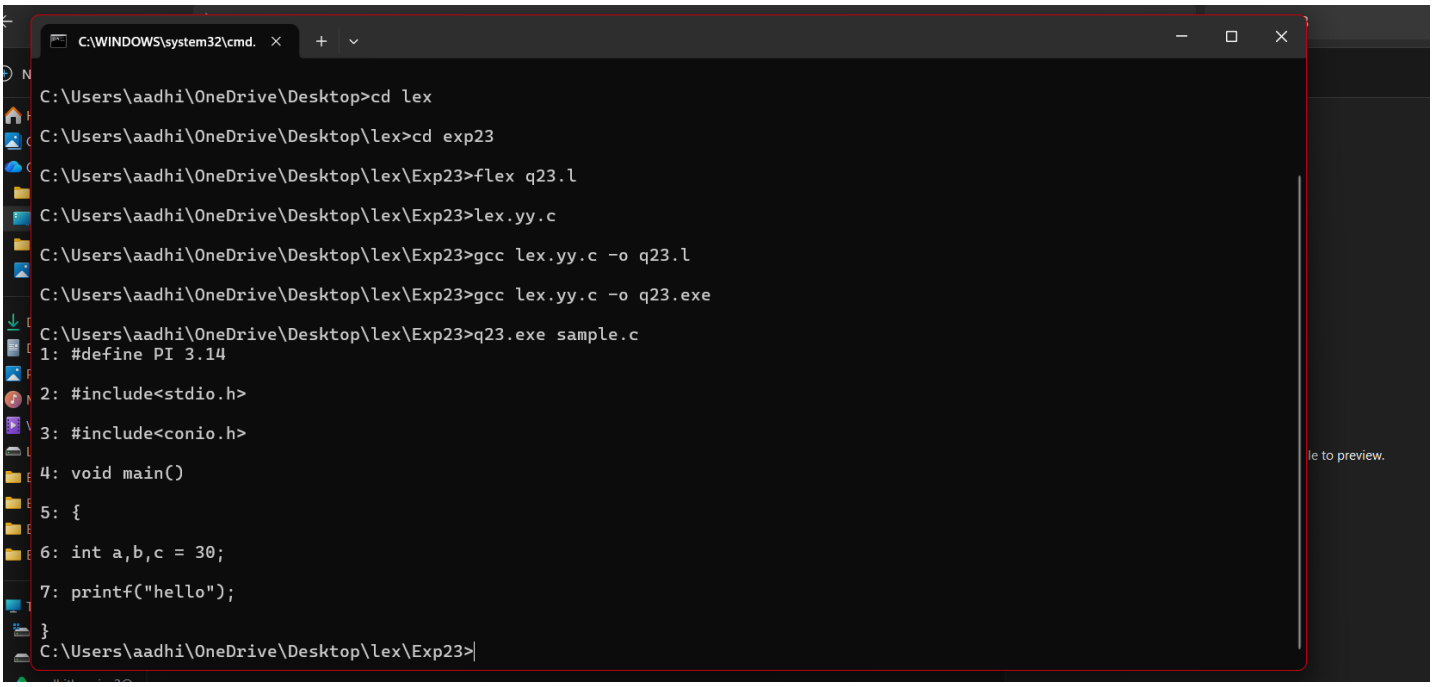
```
%{
#include <stdio.h>
int comment_lines = 0;
FILE *out_file;
%}

%option noyywrap

%%

"//".*                          { comment_lines++; }
"/*"([^*]|\*+[^/*])*\*+"/"      {
    for(int i=0; yytext[i] != '\0'; i++) {
        if(yytext[i] == '\n') comment_lines++;
    }
    comment_lines++;
}
.|\n                            { fputs(yytext, out_file); }

%%

int main(int argc, char **argv) {
    if(argc > 1) {
        yyin = fopen(argv[1], "r");
    }
    out_file = fopen("clean_output.c", "w");
    yylex();
    printf("Total comment lines eliminated: %d\n", comment_lines);
    return 0;
}
```

Sample Input:

```
#define PI 3.14
#include<stdio.h>
#include<conio.h>
void main()
{
int a,b,c = 30;
```

```
printf("hello");
}
```

Output:



24. Write a LEX program to count the number of comment lines in a given C program and eliminate them and write into another file.

Input Source File: (input.c)

```
#include<stdio.h>
int main()
{
int a,b,c; /*varible declaration*/
printf("enter two numbers");
scanf("%d %d",&a,&b);
c=a+b;//adding two numbers
printf("sum is %d",c);
return 0;
}
```

Aim:

To write a LEX program to count and remove comments from a C program.

Short Procedure:

1. Read the input file.
2. Identify single-line and multi-line comments.

3. Count the comments.
4. Remove comments while copying.
5. Write the output to another file.

Code:

```
%{
#include <stdio.h>
int comment_count = 0;
%}

%%

"/*"([^*]|\*+[^/*])*\*+"/"     { comment_count++; /* Match multi-line comments */ }
"//".*                         { comment_count++; /* Match single-line comments */ }
.|\n                           { fprintf(yyout, "%s", yytext); /* Copy non-comment characters to
output */ }

%%

int yywrap() {
    return 1;
}

int main(int argc, char *argv[]) {
    if (argc < 3) {
        printf("Usage: %s <input_file.c> <output_file.c>\n", argv[0]);
        return 1;
    }

    yyin = fopen(argv[1], "r");
    if (!yyin) {
        printf("Error: Cannot open input file %s\n", argv[1]);
        return 1;
    }

    yyout = fopen(argv[2], "w");
    if (!yyout) {
        printf("Error: Cannot create output file %s\n", argv[2]);
        fclose(yyin);
        return 1;
    }

    yylex();
```

```
    printf("Total comment lines eliminated: %d\n", comment_count);

    fclose(yyin);
    fclose(yyout);
    return 0;
}
```

Sample Input:

```
#include<stdio.h>
int main()
{
int a,b,c; /*variable declaration*/
printf("enter two numbers");
scanf("%d %d",&a,&b);
c=a+b;//adding two numbers
printf("sum is %d",c);
return 0;
}
```

Output:

```
(c) Microsoft Corporation. All rights reserved.

C:\Users\aadhi>cd onedrive

C:\Users\aadhi\OneDrive>cd desktop

C:\Users\aadhi\OneDrive\Desktop>cd lex

C:\Users\aadhi\OneDrive\Desktop\lex>cd exp24

C:\Users\aadhi\OneDrive\Desktop\lex\Exp24>flex exp24.l

C:\Users\aadhi\OneDrive\Desktop\lex\Exp24>gcc lex.yy.c -o exp24

C:\Users\aadhi\OneDrive\Desktop\lex\Exp24>./exp24 sample.c output.c
'.' is not recognized as an internal or external command,
operable program or batch file.

C:\Users\aadhi\OneDrive\Desktop\lex\Exp24>./exp24 sample.c output.c
'.' is not recognized as an internal or external command,
operable program or batch file.

C:\Users\aadhi\OneDrive\Desktop\lex\Exp24>exp24.exe sample.c output.c
Total comment lines eliminated: 2

C:\Users\aadhi\OneDrive\Desktop\lex\Exp24>
```

25. Write a LEX program to identify the capital words from the given input.

Aim:

To write a LEX program to identify and display capital words from the given input.

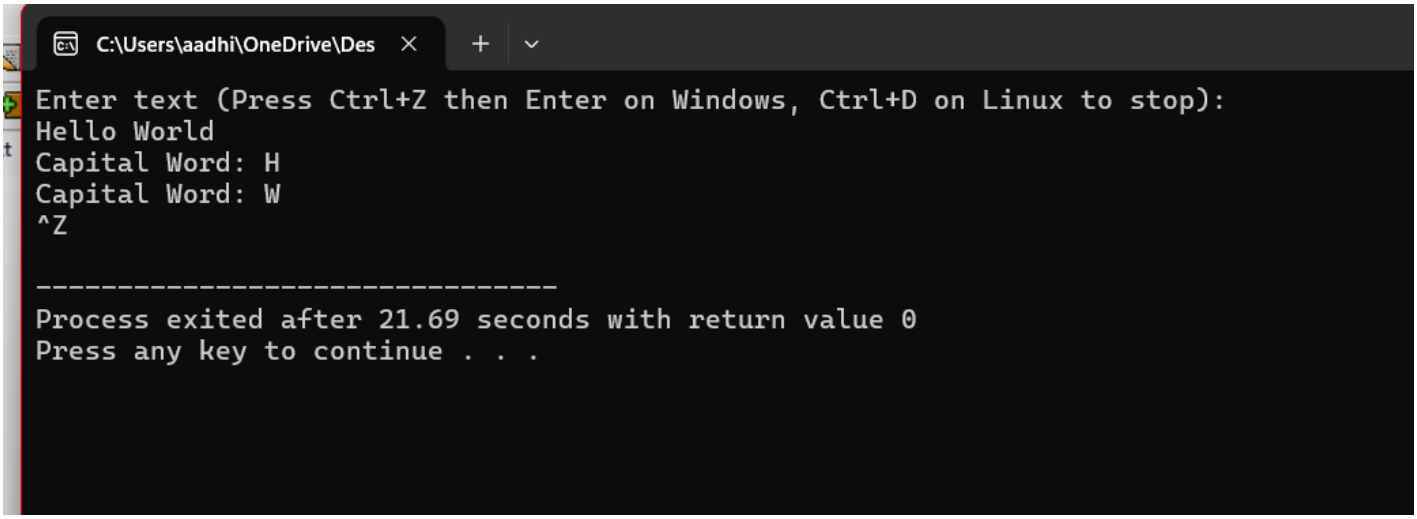
Short Procedure:

1. Create a LEX file with the .l extension.
2. Define a pattern to recognize words containing only capital letters (A–Z).
3. Write the corresponding action to display the identified capital words.
4. Save the LEX program and compile it using flex.
5. Compile the generated C file using gcc.
6. Run the program and enter the input text.
7. The program identifies and displays all the capital words from the input.

Code:

```
%{
#include <stdio.h>
%}

%%
[A-Z]+    { printf("Capital Word: %s\n", yytext); }
.|\n      { /* Ignore all other characters */ }
%%

int yywrap() {
   return 1;
}

int main() {
   printf("Enter text (Press Ctrl+Z then Enter on Windows, Ctrl+D on Linux to stop):\n");
   yylex();
   return 0;
}
```

Output:



26. Write a LEX Program to check the email address is valid or not.

Aim:

To write a LEX program to check whether a given email address is valid or not.

Short Procedure:

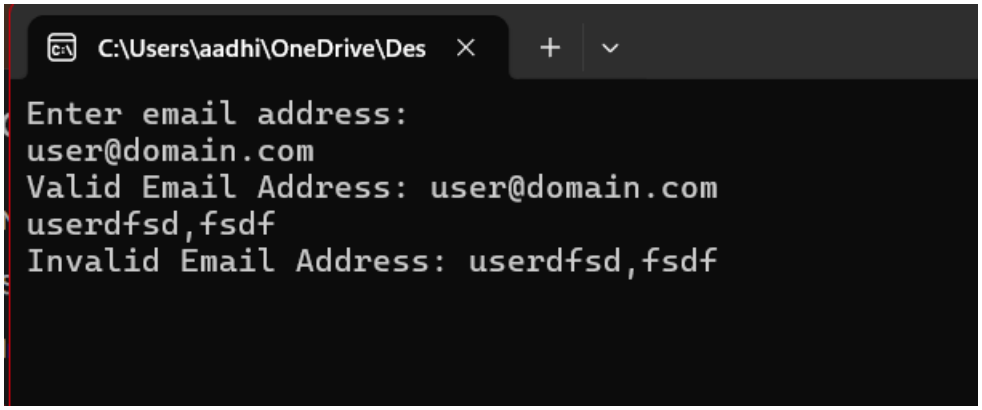
1. Create a LEX file with the .l extension.
2. Define a regular expression pattern to recognize a valid email structure (username@domain.extension).
3. Write the corresponding actions to set a flag or display whether the email is valid or invalid.
4. Save the LEX program and compile it using flex.
5. Compile the generated lex.yy.c file using gcc.
6. Run the executable and enter the input email address.
7. The program evaluates the input against the pattern and displays the result.

Code:

```
%{
#include <stdio.h>
%}

%%
[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}     { printf("Valid Email Address: %s\n", yytext); }
.+                                  { printf("Invalid Email Address: %s\n", yytext); }
\n                                  { /* Ignore newlines */ }
%%
```

```
int yywrap() {
    return 1;
}

int main() {
    printf("Enter email address:\n");
    yylex();
    return 0;
}
```

Output:



27. Write a LEX Program to convert the substring abc to ABC from the given input string.

Aim:

To write a LEX program that converts all occurrences of the substring abc to ABC in a given input string.

Short Procedure:

1. Create a LEX file with the .l extension.
2. Define a pattern to match the specific substring abc.
3. Write an action for the pattern abc to print ABC.
4. Include a default rule (.|\n) to echo all other characters without changes.
5. Save the LEX file and compile it using flex.
6. Compile the generated lex.yy.c file using gcc.
7. Run the executable, enter the input string, and observe the modified output.

Code:

```
%{
#include <stdio.h>
%}
```

```
%%
"abc"    { printf("ABC"); }
.|\n     { printf("%s", yytext); }
%%

int yywrap() {
   return 1;
}

int main() {
   printf("Enter string to modify:\n");
   yylex();
   return 0;
}
```

Output:

```
C:\Users\aadhi\OneDrive\Desktop\lex\Exp27>exp27.exe
Enter string to modify:
aadhi
aadhi
abc
ABC
^Z

C:\Users\aadhi\OneDrive\Desktop\lex\Exp27>exp27.exe
Enter string to modify:
abc
ABC
```

28. The Company ABC runs with employees with several departments. The Organization manager had all the mobile numbers of employees. Assume that you are the manager and need to verify the valid mobile numbers because there may be some invalid numbers present. Implement a LEX program to check whether the mobile number is valid or not.

Aim:

To write a LEX program to check whether a given mobile number is valid or invalid.

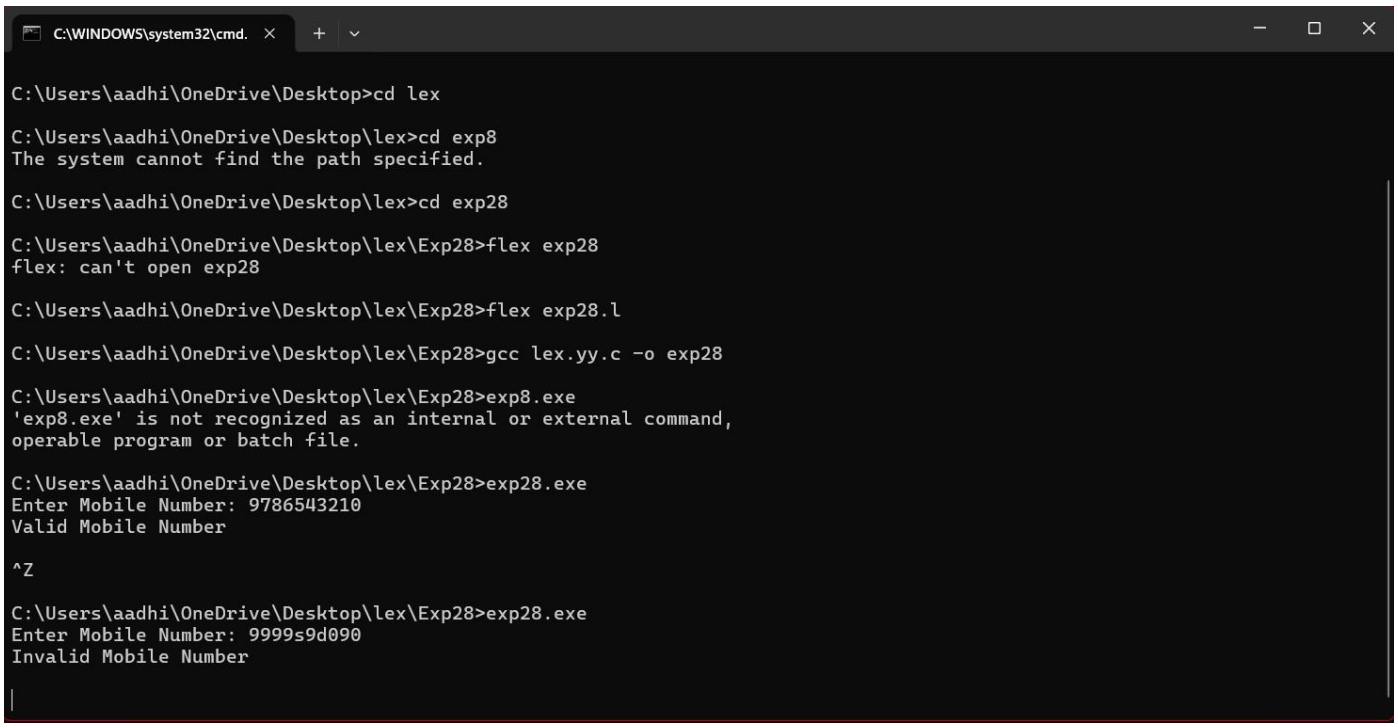
Short Procedure:

1. Create a LEX file with a pattern for a valid mobile number.
2. Accept the mobile number as input.
3. Check whether it contains exactly 10 digits and starts with 6, 7, 8, or 9.
4. Display whether the mobile number is valid or invalid.
5. Compile and execute the LEX program.

Code:

```
%{
#include <stdio.h>
%}

%%
[6-9][0-9]{9}     { printf("Valid Mobile Number\n"); }
[0-9]+            { printf("Invalid Mobile Number\n"); }
.*                { printf("Invalid Mobile Number\n"); }
%%

int yywrap()
{
   return 1;
}

int main()
{
   printf("Enter Mobile Number: ");
   yylex();
   return 0;
}
```

Output:



29. Implement Lexical Analyzer using LEX or FLEX (Fast Lexical Analyzer). The program should separate the tokens in the given C program and display them with the appropriate caption.

Aim:

To implement a lexical analyzer using LEX/FLEX to identify and display different tokens present in a C program with appropriate captions.

Short Procedure:

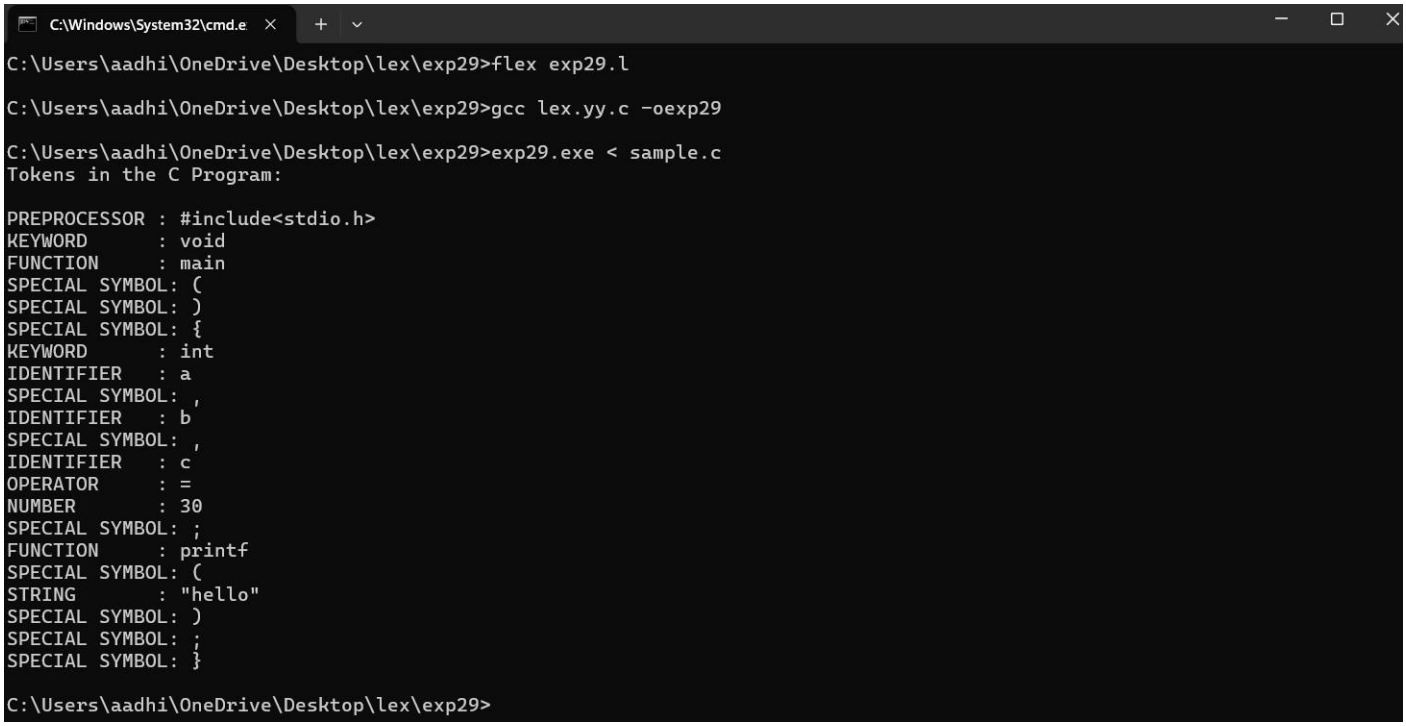
1. Create a LEX program containing patterns for keywords, identifiers, numbers, operators, strings, and special symbols.
2. Give the C source program as input.
3. The lexical analyzer scans the source program character by character.
4. It identifies each token and displays its corresponding type.
5. Compile and execute the program using FLEX and GCC.

Code:

```
%{
#include <stdio.h>
%}

%%
"#include<stdio.h>" { printf("PREPROCESSOR : %s\n", yytext); }

"void"|"int"|"char"|"float" { printf("KEYWORD      : %s\n", yytext); }

"main"                  { printf("FUNCTION     : %s\n", yytext); }

[a-zA-Z_][a-zA-Z0-9_]*      { printf("IDENTIFIER   : %s\n", yytext); }

[0-9]+                  { printf("NUMBER       : %s\n", yytext); }

\"[^\"]*\"              { printf("STRING       : %s\n", yytext); }

"="|"+"|"-"|"*"|"/"         { printf("OPERATOR     : %s\n", yytext); }

"(" | ")" | "{" | "}" | ";" | ","
                    { printf("SPECIAL SYMBOL: %s\n", yytext); }

[ \t\n]+                { /* Ignore whitespace */ }
```

```
.                       { printf("UNKNOWN       : %s\n", yytext); }
%%

int yywrap()
{
   return 1;
}

int main()
{
   printf("Tokens in the C Program:\n\n");
   yylex();
   return 0;
}
```

Output:



Exp30:

Aim:

To write a LEX program to identify and count the number of consonants present in a given word or sentence.

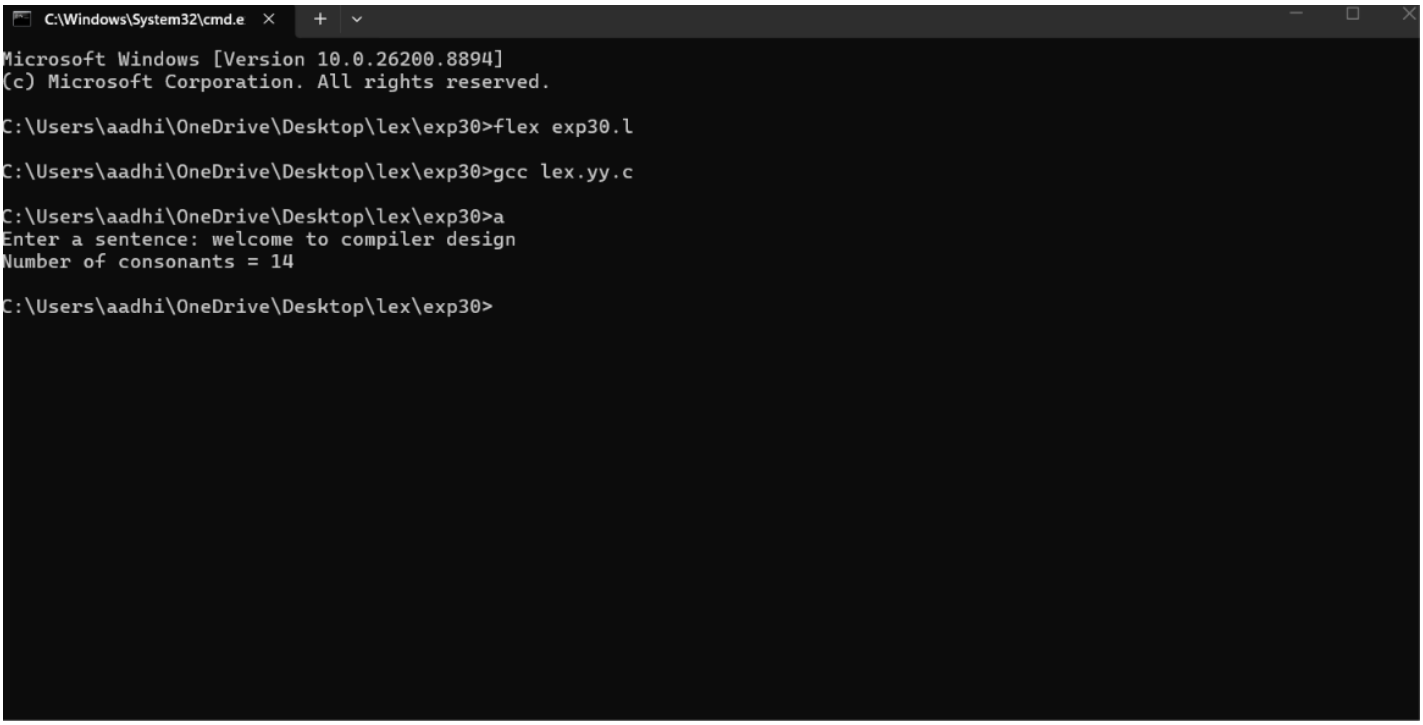
Short Procedure:

1. Create a LEX program with patterns for alphabetic characters.
2. Check each character in the input.
3. If the character is an alphabet and is not a vowel, count it as a consonant.
4. Ignore spaces, digits, and special characters.
5. Display the total number of consonants.

Code:

```
%{
#include <stdio.h>

int count = 0;
%}

%%
[bcdfghjklmnpqrstvwxyzBCDFGHJKLMNPQRSTVWXYZ]  { count++; }
.                                             { }
\n                                             { return 0; }
%%

int yywrap()
{
   return 1;
}

int main()
{
   printf("Enter a sentence: ");
   yylex();

   printf("Number of consonants = %d\n", count);

   return 0;
}
```

Output:



31. Keywords are predefined, reserved words used in programming that have special meanings to the compiler. Keywords are part of the syntax and they cannot be used as an identifier. In general, there are 32 keywords. The prime function of Lexical Analyser is token Generation. Among the 6 types of tokens, differentiating Keywords and identifiers is a challenging issue. Thus write a LEX program to separate keywords and identifiers.

Aim:

To write a **LEX program to separate keywords and identifiers** from the given input and generate the corresponding tokens.

Short Procedure:

1. Define the **keywords** and **identifier pattern** in the LEX program.
2. Read the input source code character by character.
3. Compare each word with the predefined list of keywords.
4. If the word matches a keyword, classify it as a **keyword**; otherwise, classify it as an **identifier**.
5. Compile and execute the LEX program to display the separated keywords and identifiers.

Code:

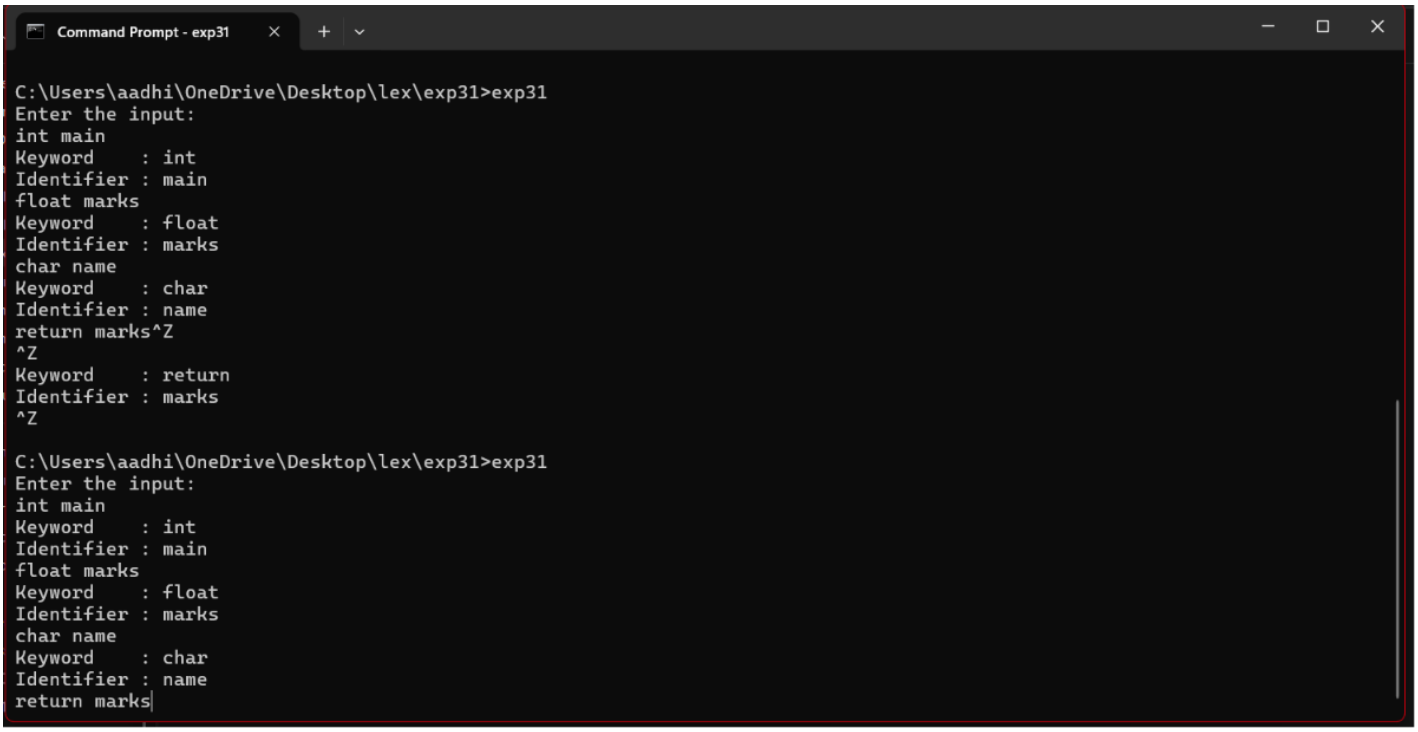
```
%{
#include <stdio.h>
%}
```

```
%%

"auto"|"break"|"case"|"char"|"const"|"continue"|"default"|"do"|"double"|"else"|"enum"|"extern"|
"float"|"for"|"goto"|"if"|"int"|"long"|"register"|"return"|"short"|"signed"|"sizeof"|"static"|"struct"
|"switch"|"typedef"|"union"|"unsigned"|"void"|"volatile"|"while"      { printf("Keyword    : %s\n",
yytext); }

[a-zA-Z_][a-zA-Z0-9_]*     { printf("Identifier : %s\n", yytext); }

[ \t\n]+    ;

.    ;

%%

int main()
{
   printf("Enter the input:\n");
   yylex();
   return 0;
}

int yywrap()
{
   return 1;
}
```

Sample Input:

```
int main
float marks
char name
return marks
```

Output:



32. Write a LEX program to identify and count positive and negative numbers.

Aim:

To write a LEX program to identify and count positive and negative numbers from the given input.

Short Procedure:

1. Define patterns for positive and negative numbers.
2. Read the input using the LEX analyzer.
3. Identify numbers beginning with + or without a sign as positive.
4. Identify numbers beginning with - as negative.
5. Increment the respective counters and display the total count.

Code:

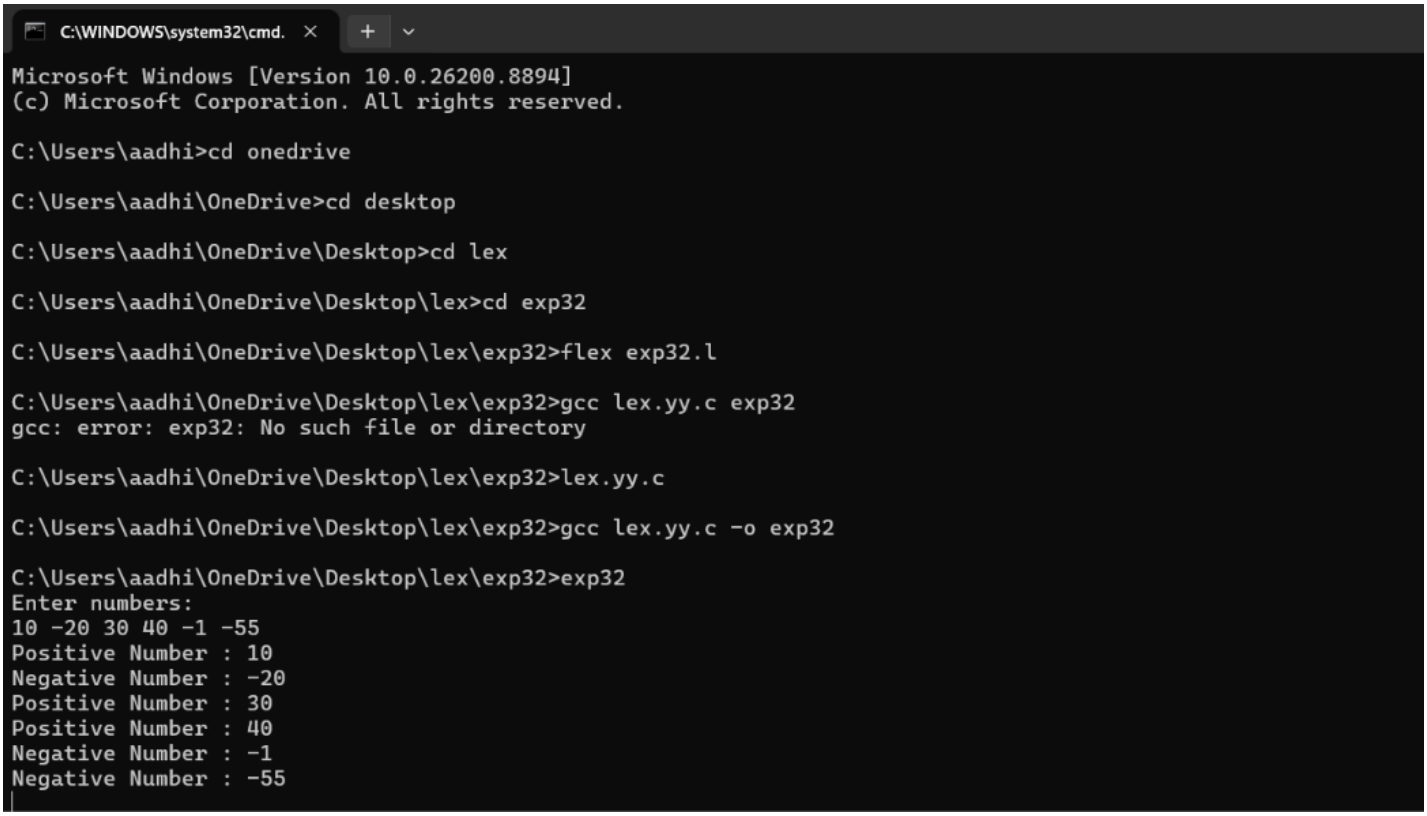
```
%{
#include <stdio.h>

int positive = 0;
int negative = 0;
```

```
%}

%%

[+]?[0-9]+(\.[0-9]+)?     {
                            printf("Positive Number : %s\n", yytext);
                            positive++;
                          }

-[0-9]+(\.[0-9]+)?        {
                            printf("Negative Number : %s\n", yytext);
                            negative++;
                          }

[ \t\n]+                    ;

.                         ;

%%

int main()
{
   printf("Enter numbers:\n");
   yylex();

   printf("\nPositive Numbers Count = %d\n", positive);
   printf("Negative Numbers Count = %d\n", negative);

   return 0;
}

int yywrap()
{
   return 1;
}
```

Sample Input:

10 -20 30 40 -1 -55

Output:



33. A networking company wants to validate the URL for its clients. Write a LEX program to implement the same.

Aim:

To write a LEX program to validate a URL and determine whether the given URL is valid or invalid.

Short Procedure:

1. Define a regular expression for a valid URL.
2. Read the URL using the LEX analyzer.
3. Match the input against the URL pattern.
4. If it matches, display **Valid URL**.
5. Otherwise, display **Invalid URL**.

Code:

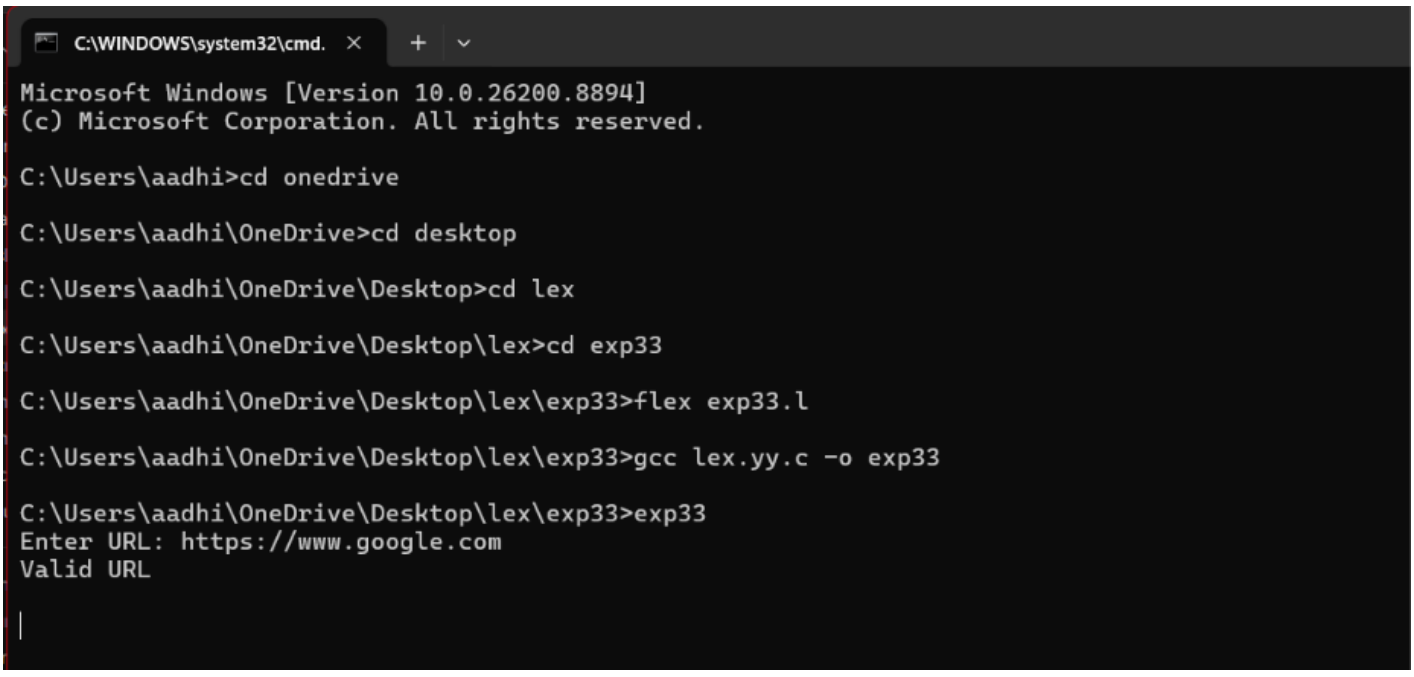
```
%{
#include <stdio.h>
%}

%%

https?://[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}(/[^ \t\n]*)? {
```

```
    printf("Valid URL : %s\n", yytext);
}

[^\n]+ {
    printf("Invalid URL : %s\n", yytext);
}

\n ;

%%

int main()
{
    printf("Enter URL:\n");
    yylex();
    return 0;
}

int yywrap()
{
    return 1;
}
```

Sample Input:

https://www.google.com

Output:



34. School management wants to validate the DOB of all students. Write a LEX program to implement it.

Aim:

To write a LEX program to validate the Date of Birth (DOB) of students in the format DD/MM/YYYY.

Short procedure:

1. Define a regular expression for the DOB format DD/MM/YYYY.
2. Accept the day from 01–31.
3. Accept the month from 01–12.
4. Accept a four-digit year.
5. If the input matches the pattern, display Valid DOB; otherwise display Invalid DOB.

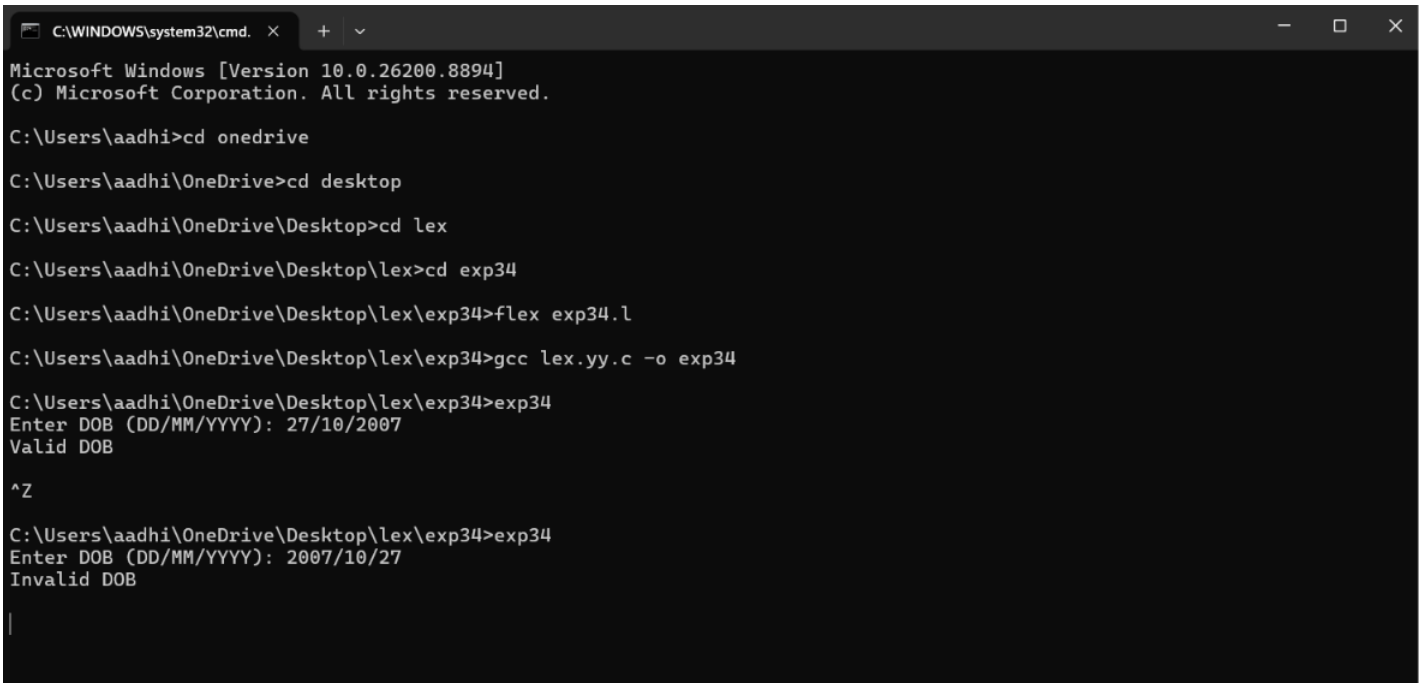
Code:

```
%{
#include <stdio.h>
%}

%%

(0[1-9]|[12][0-9]|3[01])[/](0[1-9]|1[0-2])[/][0-9]{4}      { printf("Valid DOB\n"); }

.*                                          { printf("Invalid DOB\n"); }

%%

int main()
{
   printf("Enter DOB (DD/MM/YYYY): ");
   yylex();
   return 0;
}

int yywrap()
{
   return 1;
}
```

Sample Input:

27/10/2007

Output:



35. Write a LEX program to check whether the given input is a digit or not.

Aim:

To write a LEX program to check whether the given input is a digit or not.

Short Procedure:

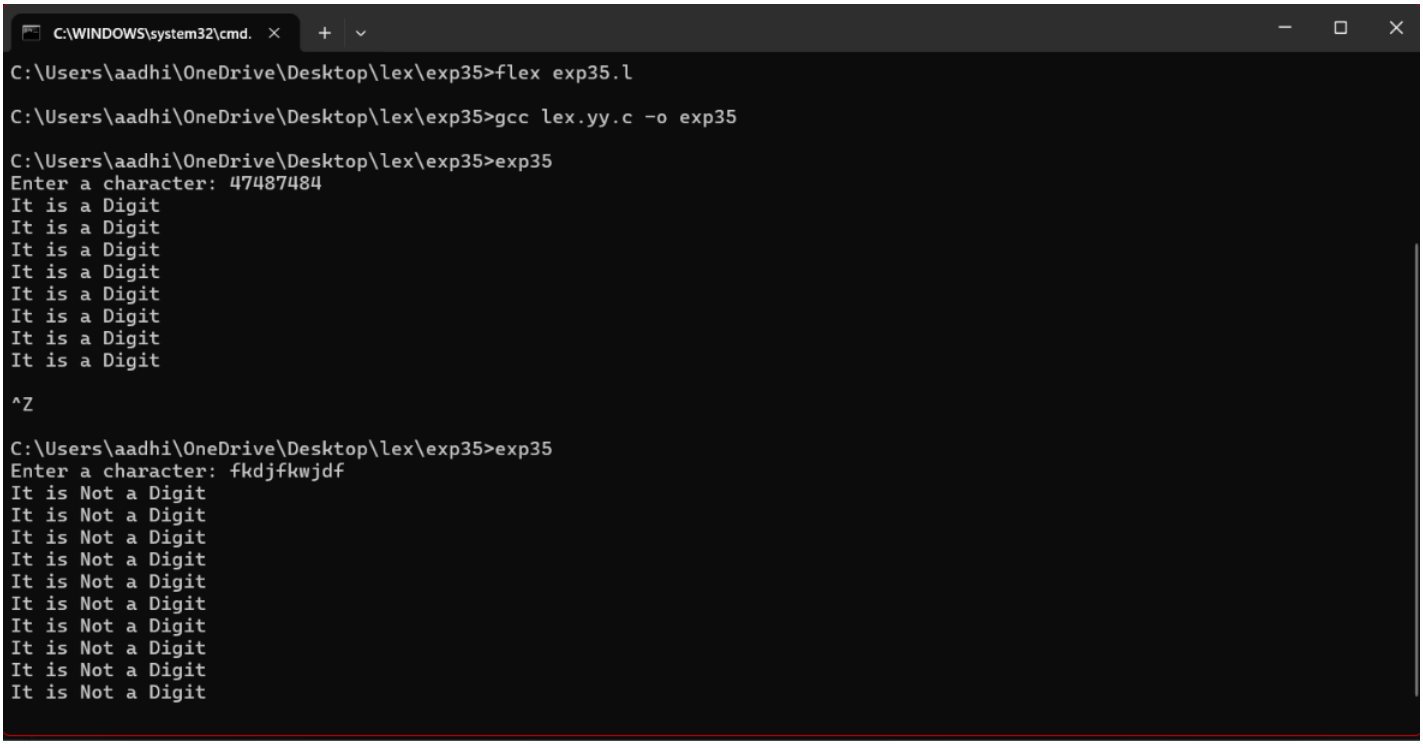
1. Define the pattern [0-9] to recognize a digit.
2. Read the input character using LEX.
3. If the character matches [0-9], display It is a Digit.
4. Otherwise, display It is Not a Digit.

Code:

```
%{
#include <stdio.h>
%}

%%

[0-9]       { printf("It is a Digit\n"); }
```

```
.           { printf("It is Not a Digit\n"); }

%%

int main()
{
   printf("Enter a character: ");
   yylex();
   return 0;
}

int yywrap()
{
   return 1;
}
```

Output:



36. A School student was asked to do basic mathematical operations. Implement a LEX program to implement the same.

Aim:

To write a LEX program to perform basic mathematical operations such as addition, subtraction, multiplication, and division.

Short Procedure:

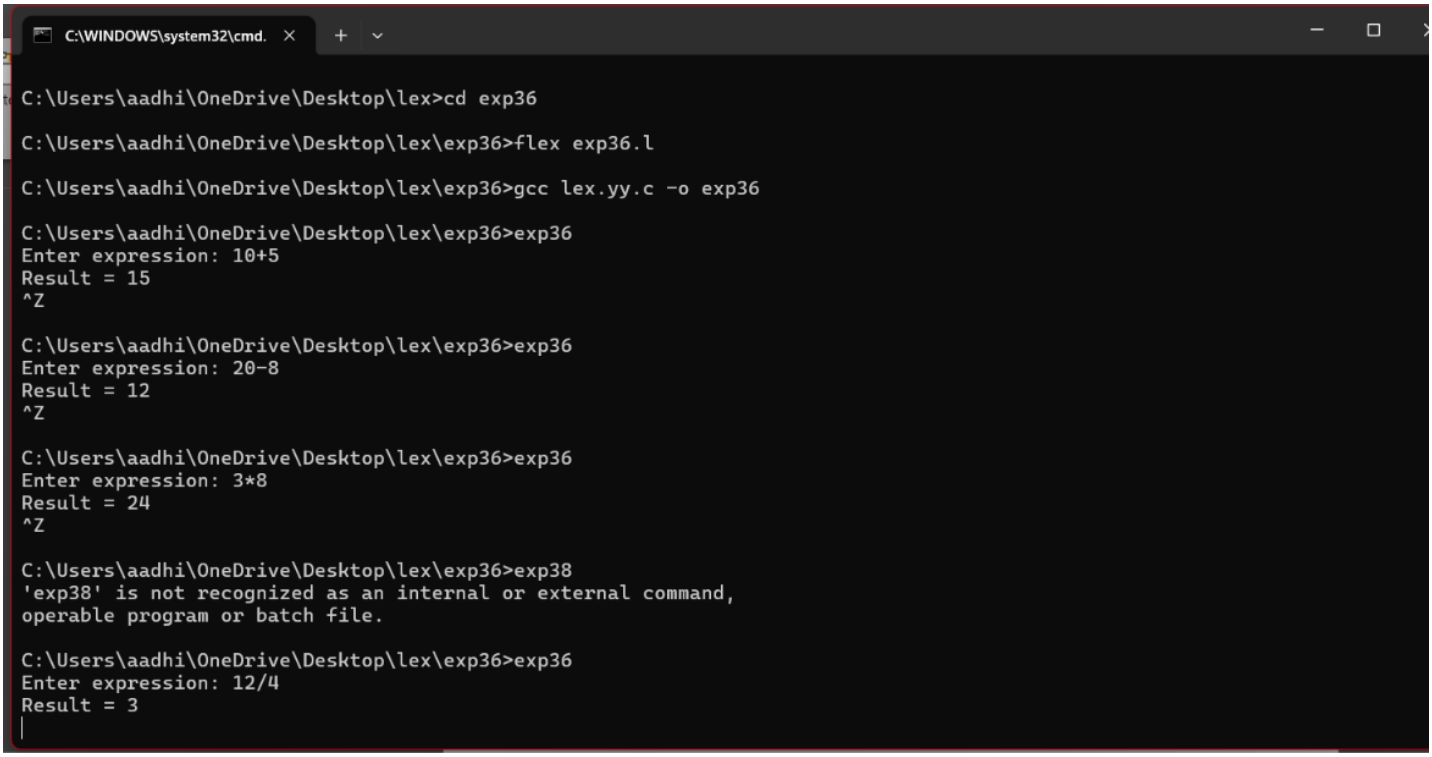
1. Define patterns for numbers and mathematical operators.
2. Read the arithmetic expression using LEX.
3. Identify the operator (+, -, *, /).
4. Perform the corresponding mathematical operation.
5. Display the calculated result.

Code:

```
%{
#include <stdio.h>
#include <stdlib.h>

int num1, num2;
char op;
%}

%%

[0-9]+[ \t]*[+\-*/][ \t]*[0-9]+ {
    sscanf(yytext, "%d %c %d", &num1, &op, &num2);

    if(op == '+')
        printf("Result = %d\n", num1 + num2);
    else if(op == '-')
        printf("Result = %d\n", num1 - num2);
    else if(op == '*')
        printf("Result = %d\n", num1 * num2);
    else if(op == '/')
    {
        if(num2 != 0)
            printf("Result = %d\n", num1 / num2);
        else
            printf("Cannot divide by zero\n");
    }
}

[ \t\n]+ ;

. {
    printf("Invalid Input\n");
}
```

```
%%

int main()
{
   printf("Enter expression: ");
   yylex();
   return 0;
}

int yywrap()
{
   return 1;
}
```

Output:



37. Write a Lex program to count the frequency of the given word in a given sentence.

Aim:

To write a LEX program to count the frequency of a given word in a sentence.

Short Procedure:

1. Read the word whose frequency needs to be counted.
2. Read the sentence using LEX.

3. Identify each word from the sentence.
4. Compare each word with the given word.
5. Increment the counter whenever a match is found and display the frequency.

Code:

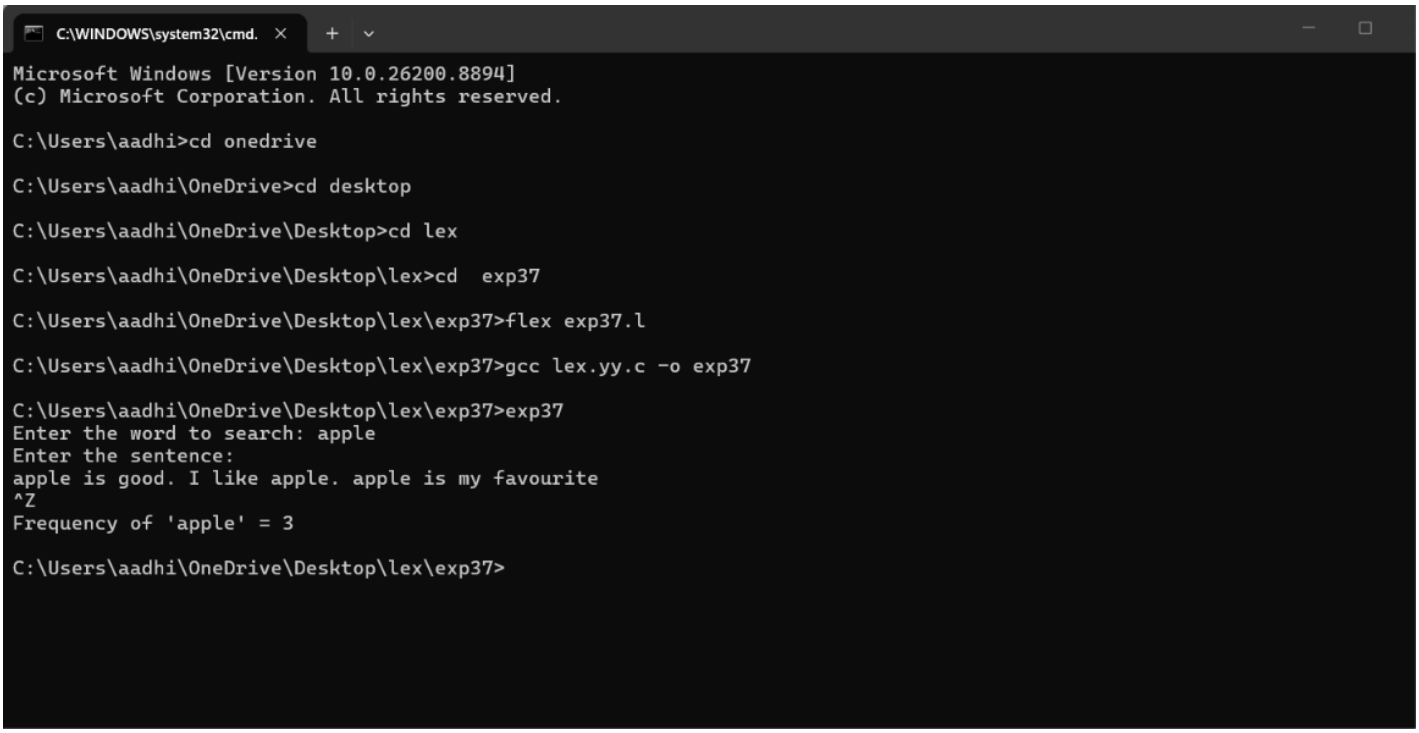
```
%{
#include <stdio.h>
#include <string.h>

char word[100];
int count = 0;
%}

%%

[a-zA-Z]+ {
   if(strcmp(yytext, word) == 0)
      count++;
}

[ \t\n]+ ;

. ;

%%

int main()
{
   printf("Enter the word to search: ");
   scanf("%s", word);

   printf("Enter the sentence:\n");
   getchar();

   yylex();

   printf("Frequency of '%s' = %d\n", word, count);

   return 0;
}

int yywrap()
{
```

```
    return 1;
}
```

Sample Input:

Apple.
Apple is good. I like apple. Apple is my favourite.

Output:



38. Write a Lex program to find the length of the longest word.

Aim:

To write a LEX program to find the length of the longest word in a given sentence.

Short Procedure:

1. Read the sentence using LEX.
2. Identify each word using [a-zA-Z]+.
3. Find the length of each word using strlen().
4. Compare each word length with the current maximum.
5. Store and display the longest word and its length.

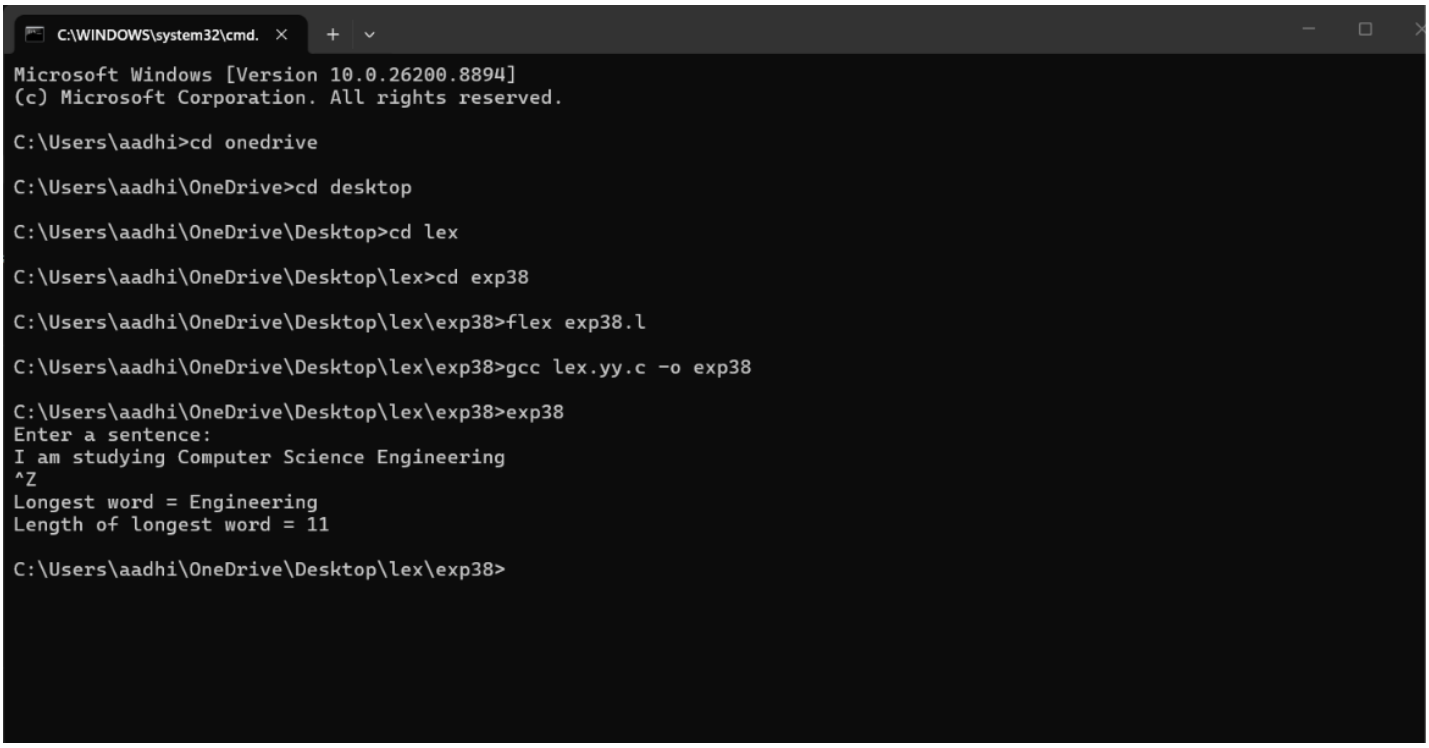
Code:

```
%{
#include <stdio.h>
#include <string.h>

int max = 0;
char longest[100];
%}

%%

[a-zA-Z]+ {
   if(strlen(yytext) > max)
   {
      max = strlen(yytext);
      strcpy(longest, yytext);
   }
}

[ \t\n]+ ;

. ;

%%

int main()
{
   printf("Enter a sentence:\n");
   yylex();

   printf("Longest word = %s\n", longest);
   printf("Length of longest word = %d\n", max);

   return 0;
}

int yywrap()
{
   return 1;
}
```

Sample Input:

I am studying in Computer Science Engineering

Output:



39. Write a Lex code to replace a word with another word in a file.

Aim:

To write a LEX program to replace a given word with another word in a file.

Short Procedure:

1. Read the old word and the new word.
2. Read the file content using LEX.
3. Compare each word with the old word.
4. If it matches, print the new word.
5. Otherwise, print the original word.

Code:

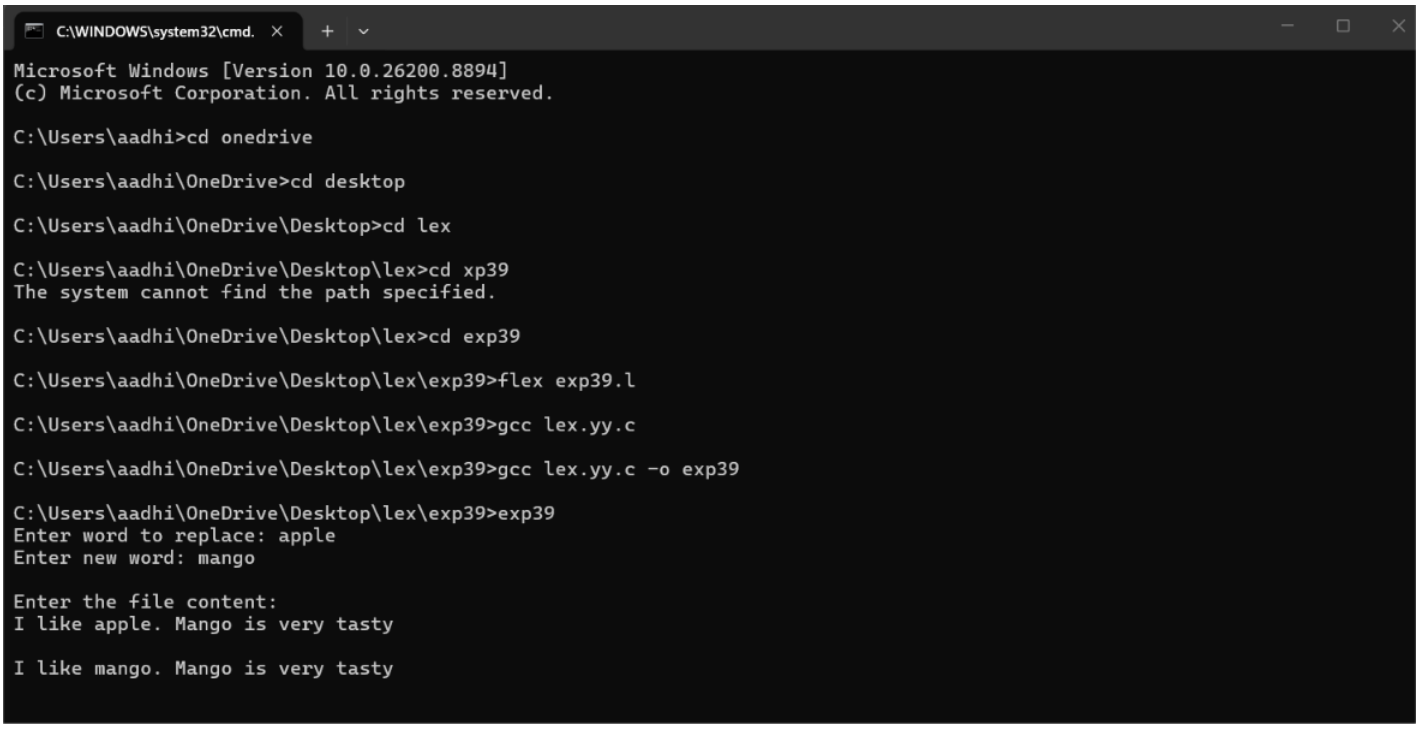
```
%{
#include <stdio.h>
#include <string.h>

char oldword[100];
char newword[100];
%}
```

```
%%

[a-zA-Z]+ {
   if(strcmp(yytext, oldword) == 0)
      printf("%s", newword);
   else
      printf("%s", yytext);
}

[ \t\n]+ {
   printf("%s", yytext);
}

. {
   printf("%s", yytext);
}

%%

int main()
{
   printf("Enter word to replace: ");
   scanf("%s", oldword);

   printf("Enter new word: ");
   scanf("%s", newword);

   printf("\nEnter the file content:\n");

   yylex();

   return 0;
}

int yywrap()
{
   return 1;
}
```

Sample Input:

```
Apple
Mango
I like apple. Mango is very tasty.
```

Output:



40. Write a FLEX (Fast Lexical Analyzer) program that should perform the token separation for the given C program and display with appropriate caption.

Aim:

To write a FLEX program to perform token separation for a given C program and display the tokens with appropriate captions such as Keywords, Identifiers, Numbers, Operators, Special Symbols, and String Literals.

Short Procedure:

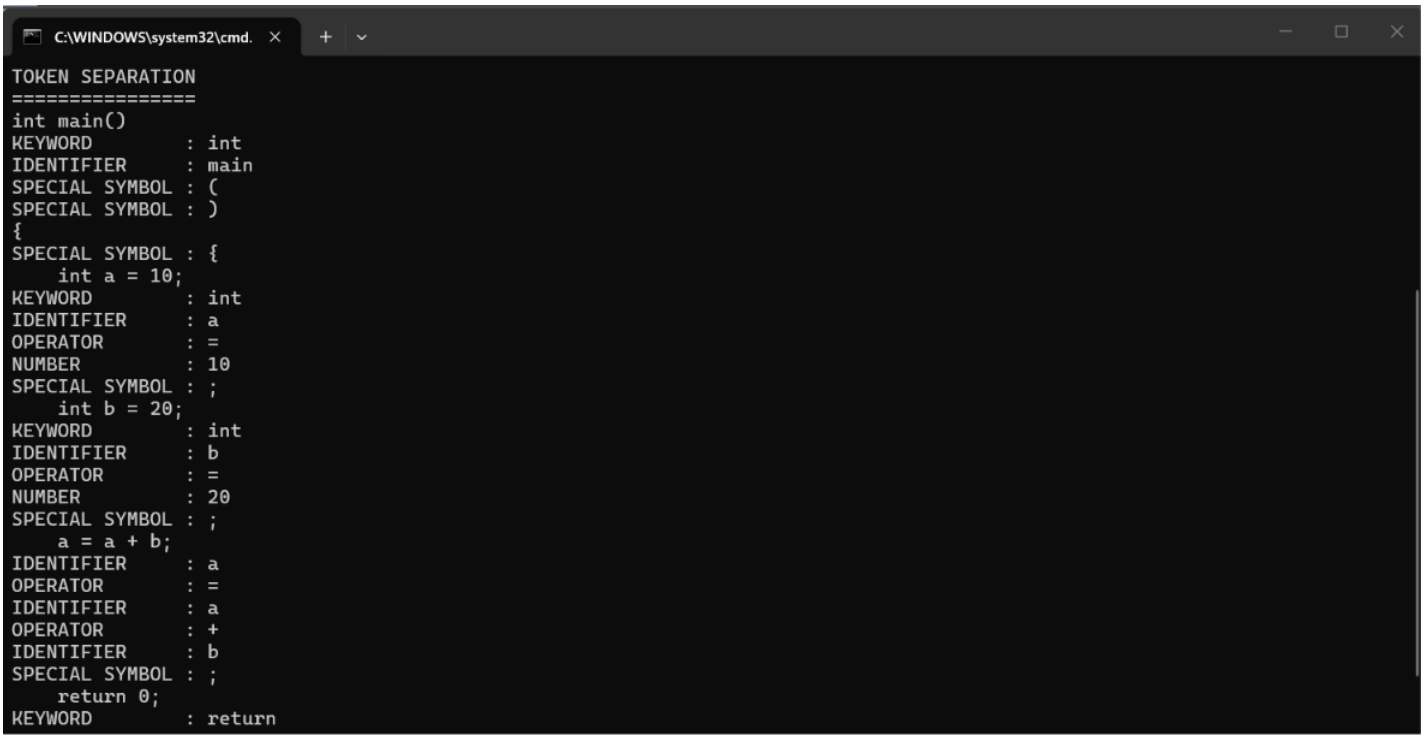
1. Define regular expressions for different C tokens.
2. Read the given C program using FLEX.
3. Identify keywords, identifiers, numbers, operators, strings, and special symbols.
4. Display each token with its appropriate caption.
5. Ignore spaces, newlines, and comments.

Code:

```
%{
#include <stdio.h>
%}

%%
```

```
"int"|"float"|"char"|"double"|"if"|"else"|"for"|"while"|"return"|"void"|"break"|"continue"          {
printf("KEYWORD        : %s\n", yytext); }

[a-zA-Z_][a-zA-Z0-9_]* { printf("IDENTIFIER     : %s\n", yytext); }

[0-9]+ { printf("NUMBER         : %s\n", yytext); }

"=="|"!="|"<="|">="|"++"|"--" { printf("OPERATOR        : %s\n", yytext); }

"+"|"-"|"*"|"/"|"="|"<"|">"|"%" { printf("OPERATOR        : %s\n", yytext); }

"("|")"|"{"|"}"|"["|"]"|";"|"," { printf("SPECIAL SYMBOL : %s\n", yytext); }

[ \t\n]+ ;

. { printf("UNKNOWN         : %s\n", yytext); }

%%

int main()
{
   printf("TOKEN SEPARATION\n");
   printf("================\n");
   yylex();
   return 0;
}

int yywrap()
{
   return 1;
}
```

Output:


C:\WINDOWS\system32\cmd.
TOKEN SEPARATION
==========
int main()
KEYWORD : int
IDENTIFIER : main
SPECIAL SYMBOL : (
SPECIAL SYMBOL :)
{
SPECIAL SYMBOL : {
int a = 10;
KEYWORD : int
IDENTIFIER : a
OPERATOR : =
NUMBER : 10
SPECIAL SYMBOL : ;
int b = 20;
KEYWORD : int
IDENTIFIER : b
OPERATOR : =
NUMBER : 20
SPECIAL SYMBOL : ;
a = a + b;
IDENTIFIER : a
OPERATOR : =
IDENTIFIER : a
OPERATOR : +
IDENTIFIER : b
SPECIAL SYMBOL : ;
return 0;
KEYWORD : return